

Alur Belajar Project NaikApa

Dokumen ini berisi urutan belajar untuk memahami project Android **NaikApa** dari dasar Android Studio sampai bagian inti seperti database, API, peta, routing, dan sistem rekomendasi.

Target akhirnya: kamu bisa membaca struktur project, memahami alur kode, menjelaskan cara kerja aplikasi, menjalankan/debug aplikasi, lalu mulai menambah atau memperbaiki fitur sendiri.

1. Gambaran Besar Project

Sebelum masuk ke kode, pahami dulu aplikasi ini dibuat untuk apa.

NaikApa adalah aplikasi Android native Kotlin untuk rekomendasi transportasi multimoda di Jabodetabek. Aplikasi ini menggabungkan:

- Login dan register lokal.
- Pencarian lokasi.
- Data transportasi umum GTFS.
- Rute kendaraan pribadi via TomTom API.
- Peta interaktif dengan osmdroid.
- Sistem rekomendasi berdasarkan waktu, biaya, jalan kaki, transit, dan gangguan.
- Riwayat perjalanan, favorit, profil, dan laporan gangguan.

File yang perlu dibaca:

- README.md
- konsep/prd_naik_apa_mvp.md
- konsep/KONSEP_TEKNIS.md

Yang harus bisa kamu jawab setelah tahap ini:

- Aplikasi ini menyelesaikan masalah apa?
 - Fitur utamanya apa saja?
 - Data apa saja yang dipakai?
 - Bagian mana yang online dan offline?
-

2. Dasar Android Studio dan Gradle

Tahap ini penting supaya kamu tahu cara project Android disusun dan dijalankan.

Pelajari konsep:

- Android Studio project.

- Module Android, di project ini modulnya `app`.
- Gradle sebagai build system.
- `compileSdk`, `minSdk`, `targetSdk`.
- Dependency/library.
- `local.properties`.
- Build variant debug/release.

File yang perlu dibaca:

- `settings.gradle.kts`
- `build.gradle.kts`
- `app/build.gradle.kts`
- `gradle/libs.versions.toml`
- `local.properties`

Bagian penting di `app/build.gradle.kts`:

- `namespace = "com.example.naikapa"`
- `applicationId = "com.example.naikapa"`
- `minSdk = 26`
- `targetSdk = 36`
- `viewBinding = true`
- `buildConfig = true`
- Dependency Material, Navigation, Retrofit, osmdroid, Glide, Coroutines, dan testing.
- `BuildConfig.TOMTOM_API_KEY` yang diambil dari `local.properties`.

Alur `BuildConfig.TOMTOM_API_KEY`:

```

local.properties
    |
    v
app/build.gradle.kts membaca TOMTOM_API_KEY
    |
    v
buildConfigField("String", "TOMTOM_API_KEY", ...)
    |
    v
BuildConfig.TOMTOM_API_KEY tersedia di Kotlin
    |
    v
HomeFragment / Repository memakai key untuk TomTom API
  
```

Detail penting:

- `app/build.gradle.kts` membaca file `local.properties` dengan `Properties`.

- Jika `TOMTOM_API_KEY` tidak ada, project memakai placeholder `isi_api_key_kamu_di_sini`.
- `buildFeatures.buildConfig = true` wajib aktif supaya class `BuildConfig` dibuat.
- Kode Kotlin mengecek placeholder sebelum memanggil TomTom agar aplikasi bisa memberi pesan error yang jelas.
- API key tidak ditulis langsung di source code karena source code bisa masuk Git, sedangkan `local.properties` biasanya bersifat lokal.

Yang harus bisa kamu jawab:

- Apa fungsi `settings.gradle.kts`?
- Apa beda `root build.gradle.kts` dan `app/build.gradle.kts`?
- Kenapa API key tidak ditulis langsung di source code?
- Library apa saja yang dipakai project ini?

3. Struktur Folder Android

Pahami struktur standar Android lebih dulu.

Struktur utama:

```
app/src/main/
├── AndroidManifest.xml
├── java/com/example/naikapa/
├── res/
└── assets/
```

Folder penting:

- `java/com/example/naikapa/`: kode Kotlin.
- `res/layout/`: file tampilan XML.
- `res/drawable/`: icon, gambar, background.
- `res/values/`: warna, string, tema, dimensi.
- `res/navigation/`: grafik navigasi fragment.
- `assets/databases/`: database prebuilt GTFS.

File yang perlu dibaca:

- `app/src/main/AndroidManifest.xml`
- `app/src/main/res/values/strings.xml`
- `app/src/main/res/values/colors.xml`
- `app/src/main/res/values/themes.xml`

- `app/src/main/res/navigation/nav_main.xml`
- `app/src/main/res/navigation/nav_auth.xml`

Yang harus bisa kamu jawab:

- Apa fungsi `AndroidManifest.xml`?
- Apa beda `res/` dan `assets/`?
- Kenapa teks sebaiknya disimpan di `strings.xml`?
- Apa fungsi `nav_main.xml` dan `nav_auth.xml`?

4. Manifest, Permission, dan Entry Point Aplikasi

Manifest adalah file deklarasi utama aplikasi Android.

Di project ini, manifest mendefinisikan:

- Permission internet.
- Permission lokasi.
- Permission kamera dan media.
- Activity launcher.
- FileProvider untuk foto laporan gangguan.

File utama:

- `app/src/main/AndroidManifest.xml`

Activity penting:

- `presentation/splash/SplashActivity.kt`
- `presentation/auth/AuthActivity.kt`
- `MainActivity.kt`

Alur awal aplikasi:

```
User membuka aplikasi
    |
    v
SplashActivity
    |
    |-- sudah login --> MainActivity
    |
    |-- belum login --> AuthActivity
```

Detail permission modern yang dipakai:

- **INTERNET**: wajib untuk TomTom API dan tile peta online.
- **ACCESS_FINE_LOCATION**: dipakai saat user ingin memakai GPS sebagai titik asal.

- `ACCESS_COARSE_LOCATION`: deklarasi lokasi kasar sebagai fallback permission lokasi.
- `CAMERA`: dipakai saat user mengambil foto laporan gangguan dari kamera.
- `READ_MEDIA_IMAGES`: dipakai untuk memilih gambar dari galeri pada Android 13/API 33 ke atas.
- `READ_EXTERNAL_STORAGE`: fallback untuk Android 12/API 32 ke bawah, dengan `maxSdkVersion="32"`.

Kenapa permission tetap harus dicek runtime:

- Permission yang ditulis di `AndroidManifest.xml` hanya mendeklarasikan kebutuhan aplikasi.
- Permission berisiko seperti lokasi, kamera, dan media tetap perlu diminta saat runtime.
- Di project ini, lokasi diminta ketika user menekan tombol GPS di Home.
- Kamera dan galeri diminta saat user menekan tombol ambil/pilih foto di form laporan gangguan.

Alur FileProvider untuk foto:

```
ReportPhotoHelper membuat file di cache/disruption_photos
    |
    v
FileProvider membuat content Uri
    |
    v
Intent kamera menerima Uri tersebut
    |
    v
Path file disimpan ke SQLite sebagai photo_path
```

Yang harus dipahami:

- `SplashActivity` menjadi launcher activity.
- `SessionManager` menentukan user sudah login atau belum.
- `AuthActivity` menampung login/register.
- `MainActivity` menampung halaman utama dengan bottom navigation.

Yang harus bisa kamu jawab:

- Activity mana yang pertama kali dibuka?
 - Kenapa `SplashActivity` punya intent-filter `MAIN` dan `LAUNCHER`?
 - Apa fungsi permission `ACCESS_FINE_LOCATION`?
 - Apa fungsi `FileProvider`?
-

5. Activity, Fragment, dan Lifecycle

Project ini memakai kombinasi Activity dan Fragment.

Activity:

- Container layar besar.
- Mengatur fragment/navigation.

Fragment:

- Bagian layar yang bisa berpindah melalui Navigation Component.
- Contoh: Home, Riwayat, Profil, Login, Register.

File yang perlu dibaca:

- `app/src/main/java/com/example/naikapa/presentation/splash/SplashActivity.kt`
- `app/src/main/java/com/example/naikapa/presentation/auth/AuthActivity.kt`
- `app/src/main/java/com/example/naikapa/MainActivity.kt`
- `app/src/main/java/com/example/naikapa/presentation/auth/LoginFragment.kt`
- `app/src/main/java/com/example/naikapa/presentation/auth/RegisterFragment.kt`
- `app/src/main/java/com/example/naikapa/presentation/home/HomeFragment.kt`

Lifecycle yang perlu dipahami:

- `onCreate()`
- `onCreateView()`
- `onViewCreated()`
- `onResume()`
- `onPause()`
- `onDestroyView()`

Contoh penting:

- `SplashActivity.onCreate()` memutuskan user masuk ke `MainActivity` atau `AuthActivity`.
- `LoginFragment.onViewCreated()` memasang listener tombol login.
- `HomeFragment.onResume()` mengaktifkan ulang peta dan mengecek replay dari riwayat.
- `HomeFragment.onDestroyView()` membatalkan coroutine dan menutup database.

Yang harus bisa kamu jawab:

- Kapan memakai Activity?
 - Kapan memakai Fragment?
 - Kenapa binding Fragment harus dibuat null di `onDestroyView()`?
 - Kenapa operasi berat tidak boleh langsung dilakukan di main thread?
-

6. XML Layout dan ViewBinding

Project ini memakai XML layout, bukan Jetpack Compose.

ViewBinding membuat class binding otomatis dari XML. Contoh:

- `activity_main.xml` menjadi `ActivityMainBinding`.
- `fragment_login.xml` menjadi `FragmentLoginBinding`.
- `fragment_home.xml` menjadi `FragmentHomeBinding`.

File layout penting:

- `app/src/main/res/layout/activity_splash.xml`
- `app/src/main/res/layout/activity_auth.xml`
- `app/src/main/res/layout/activity_main.xml`
- `app/src/main/res/layout/fragment_login.xml`
- `app/src/main/res/layout/fragment_register.xml`
- `app/src/main/res/layout/fragment_home.xml`
- `app/src/main/res/layout/fragment_route_detail.xml`
- `app/src/main/res/layout/fragment_riwayat.xml`
- `app/src/main/res/layout/fragment_status_gangguan.xml`
- `app/src/main/res/layout/fragment_profil.xml`

Komponen UI yang perlu dipelajari:

- `ConstraintLayout`
- `LinearLayout`
- `TextView`
- `EditText`
- `Button`
- `MaterialButton`
- `TextInputLayout`

- RecyclerView
- BottomNavigationView
- ChipGroup
- MaterialCardView
- MapView

Yang harus bisa kamu jawab:

- Bagaimana XML layout terhubung ke Kotlin?
- Apa fungsi id pada view?
- Apa beda match_parent, wrap_content, dan constraint?
- Kenapa RecyclerView butuh Adapter?

7. Navigation Component

Project ini memakai Jetpack Navigation Component untuk berpindah antar fragment.

File navigasi:

- app/src/main/res/navigation/nav_auth.xml
- app/src/main/res/navigation/nav_main.xml

Alur auth:

```
AuthActivity
  |
  v
LoginFragment <--> RegisterFragment
```

Alur main:

```
MainActivity
  |
  v
HomeFragment
RiwayatFragment
StatusGangguanFragment
ProfilFragment
RouteDetailFragment
AddEditDisruptionReportFragment
```

File Kotlin terkait:

- MainActivity.kt
- AuthActivity.kt
- LoginFragment.kt
- RegisterFragment.kt
- HomeFragment.kt

- `RouteDetailFragment.kt`

Konsep yang perlu dipahami:

- `NavHostFragment`
- `NavController`
- `findNavController()`
- Navigation action.
- Bottom navigation yang tersambung ke `NavController`.

Yang harus bisa kamu jawab:

- Bagaimana tombol register membawa user dari login ke register?
- Bagaimana bottom navigation mengganti fragment utama?
- Kenapa detail rute bukan Activity terpisah?

8. Session dan Login Lokal

Project ini memakai login lokal, bukan login cloud.

Komponen penting:

- `common/SessionManager.kt`
- `data/local/UserDao.kt`
- `data/model/User.kt`
- `presentation/auth/LoginFragment.kt`
- `presentation/auth/RegisterFragment.kt`

Alur login:

```

User input email dan password
    |
    v
LoginFragment validasi input
    |
    v
UserDao.login(email, password)
    |
    |-- berhasil --> SessionManager.saveSession()
    |
    |-- gagal -----> tampilkan pesan error
  
```

Session disimpan di:

- `SharedPreferences`

Data user disimpan di:

- SQLite tabel `users`

Yang harus dipahami:

- `SharedPreferences` untuk data kecil seperti status login.
- `SQLite` untuk data yang lebih terstruktur.
- Validasi input email dan password.
- Logout menghapus session.

Yang harus bisa kamu jawab:

- Apa beda session dan data user?
- Kenapa `SessionManager` tidak langsung menyimpan semua data profil?
- Apa risiko menyimpan password plaintext?

Catatan penting:

Project akademik ini menyimpan password lokal. Untuk aplikasi produksi, password harus di-hash dan autentikasi sebaiknya memakai backend atau auth provider.

9. Database SQLite

Database adalah bagian penting di project ini.

Project ini memakai SQLite native, bukan Room.

File utama:

- `data/local/NaikApaDbContract.kt`
- `data/local/NaikApaDatabaseHelper.kt`
- `data/local/PrebuiltDatabaseCopier.kt`
- `data/local/UserDao.kt`
- `data/local/GtfsDao.kt`
- `data/local/HistoryDao.kt`
- `data/local/SavedTripDao.kt`
- `data/local/DisruptionReportDao.kt`
- `data/local/RouteCacheDao.kt`

Tabel utama:

- `users`
- `user_profiles`
- `saved_trips`
- `search_history`
- `route_history`

- `disruption_reports`
- `route_cache`
- `gtfs_stops`
- `gtfs_routes`
- `gtfs_trips`
- `gtfs_stop_times`

Yang perlu dipahami dari `NaikApaDbContract.kt`:

- Nama tabel.
- Nama kolom.
- `SQL CREATE TABLE`.
- Foreign key.
- Index.
- `SQL DROP TABLE`.

Yang perlu dipahami dari `NaikApaDatabaseHelper.kt`:

- `SQLiteOpenHelper`.
- `onCreate()`.
- `onUpgrade()`.
- `onConfigure()`.
- Foreign key constraints.

Yang perlu dipahami dari DAO:

- `insert`.
- `query`.
- `rawQuery`.
- `update`.
- `delete`.
- Mapping dari `Cursor` ke data class.

Relasi tabel penting:

- `user_profiles.id_user` mengarah ke `users.id_user` dan bersifat unik, sehingga satu user punya satu profil.
- `saved_trips.id_user`, `search_history.id_user`, `route_history.id_user`, dan `disruption_reports.id_user` mengarah ke user yang membuat data tersebut.

- Tabel GTFS saling terhubung lewat `gtfs_routes`, `gtfs_trips`, dan `gtfs_stop_times`.
- `gtfs_stop_times.trip_id` mengarah ke trip, sedangkan `gtfs_stop_times.stop_id` mengarah ke stop.
- Foreign key diaktifkan di database helper agar data turunan ikut konsisten saat data induk dihapus.

Alasan index dibuat:

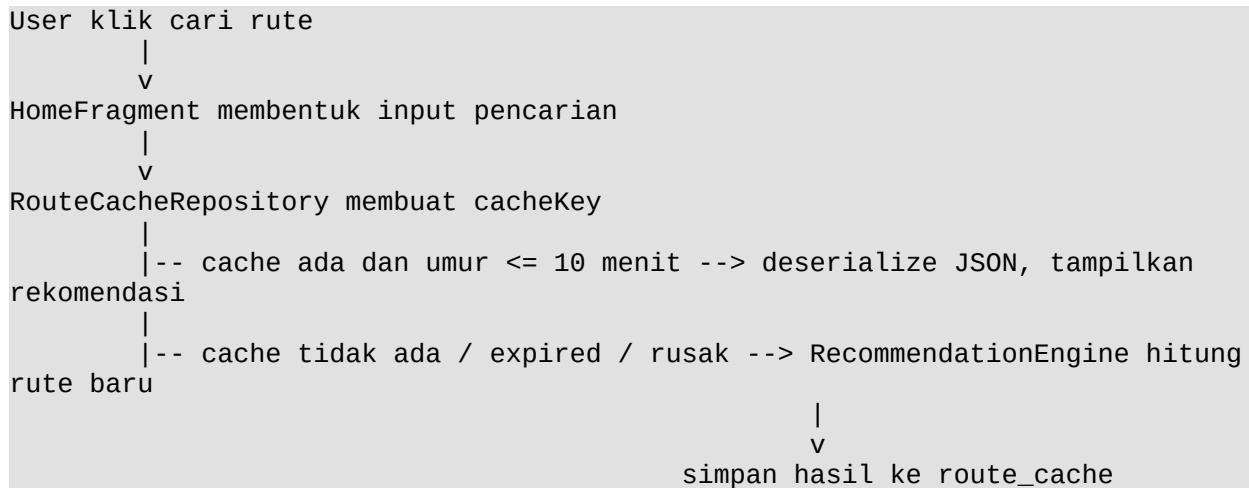
- `idx_users_email`: mempercepat pencarian user saat login.
- `idx_saved_trips_user`: mempercepat daftar favorit per user.
- `idx_search_history_user_time`: mempercepat riwayat pencarian terbaru per user.
- `idx_route_history_user_time`: mempercepat riwayat rute terbaru per user.
- `idx_disruptions_status_expired`: mempercepat pencarian laporan gangguan yang masih aktif.
- `idx_disruptions_stop_route`: mempercepat pengecekan gangguan berdasarkan stop/route yang dilewati rute.
- Index GTFS mempercepat pencarian stop, route, trip, dan koneksi stop berurutan untuk graph.
- `idx_route_cache_key`: mempercepat pencarian cache berdasarkan key unik hasil kombinasi input pencarian.
- `idx_route_cache_created_at`: mempercepat penghapusan cache lama.
- `idx_route_cache_lookup`: index pendukung untuk pencarian cache berdasarkan origin, destination, mode, dan priority.

Status `route_cache` saat ini:

- Route cache sudah terintegrasi ke flow rekomendasi utama di `HomeFragment`.
- `HomeFragment` membuat `RouteCacheRepository(RouteCacheDao(dbHelper))`.
- Sebelum `RecommendationEngine.recommend(...)` dipanggil, aplikasi mengecek cache dulu lewat `RouteCacheRepository.getRecommendation(...)`.
- Jika cache valid, hasil rekomendasi langsung dipakai tanpa hitung rute ulang.
- Jika cache miss, expired, atau JSON gagal dibaca, aplikasi menghitung rute baru lalu menyimpannya lewat `saveRecommendation(...)`.
- Cache valid selama `ROUTE_CACHE_TTL_MILLIS`, yaitu 10 menit.
- Row cache lama dibersihkan dengan `clearExpired()` berdasarkan `ROUTE_CACHE_CLEANUP_AGE_MILLIS`, yaitu 24 jam.

- Cache dengan key yang sama diganti memakai `insertOrReplace`, jadi hasil lama tidak menumpuk untuk pencarian yang identik.
- Flow chip kendaraan pribadi spesifik yang langsung memanggil routing private masih bypass cache rekomendasi ini.

Alur route cache:



Cache key dibentuk dari input yang memengaruhi hasil rute:

- Koordinat asal dan tujuan yang dibulatkan 5 desimal.
- Mode transit.
- Prioritas sorting.
- Status punya motor/mobil.
- Filter jenis kendaraan.
- Stop asal/tujuan jika lokasi berasal dari GTFS.
- Opsi hindari tol.

Koordinat dibulatkan 5 desimal supaya pencarian yang secara praktis sama tidak mudah miss karena perbedaan angka GPS sangat kecil. Di Jakarta, 5 desimal kira-kira berada pada skala sekitar 1 meter, sehingga masih cukup presisi untuk rute tetapi lebih stabil untuk cache.

Yang harus bisa kamu jawab:

- Apa fungsi DAO?
- Kenapa query database dipisah dari Fragment?
- Apa fungsi index database?
- Apa beda `readableDatabase` dan `writableDatabase`?

10. Database GTFS Prebuilt

NaikApa memakai data transportasi umum dari GTFS.

GTFS adalah format data transportasi publik yang berisi:

- Agency/operator.
- Stop/halte/stasiun.
- Route/jalur.
- Trip/perjalanan.
- Stop time/jadwal antar stop.

Folder data mentah:

- data/gtfs/
- data/gtfs-krl/
- data/gtfs-mrt/
- data/gtfs-lrt/

Database hasil olahan:

- app/src/main/assets/databases/naikapa_gtfs.db

Script terkait:

- scripts/build_gtfs_db.py
- scripts/validate_gtfs_db.py

File Android terkait:

- data/local/PrebuiltDatabaseCopier.kt
- data/local/GtfsDao.kt
- data/repository/GtfsStopSearchRepository.kt
- data/repository/NearbyTransitStopRepository.kt
- data/repository/TransitGraphRepository.kt

Alur GTFS:

```
Raw GTFS .txt
  |
  v
scripts/build_gtfs_db.py
  |
  v
naikapa_gtfs.db
  |
  v
Disimpan di assets/databases
  |
  v
Disalin ke internal database saat app pertama dibuka
  |
  v
Dibaca oleh GtfsDao
```

Yang harus bisa kamu jawab:

- Apa itu GTFS?
 - Kenapa database GTFS dibuat prebuilt?
 - Kenapa database harus disalin dari assets ke internal storage?
 - Tabel GTFS mana yang dipakai untuk membangun graph?
-

11. Model Data

Model adalah representasi data di Kotlin.

Folder:

- `app/src/main/java/com/example/naikapa/data/model/`

File penting:

- `User.kt`
- `UserProfile.kt`
- `GtfsModels.kt`
- `LocationPoint.kt`
- `SearchLocation.kt`
- `TomTomSearchModels.kt`
- `TomTomRoutingModels.kt`
- `TransitGraphModels.kt`
- `TransitRouteModels.kt`
- `CombinedRouteModels.kt`
- `RecommendationModels.kt`
- `DisruptionReport.kt`
- `SavedTrip.kt`
- `RouteHistory.kt`
- `RouteCache.kt`

Konsep yang perlu dipahami:

- `data class`.
- `enum class`.
- `sealed class`.
- Nullable type seperti `String?`.

- List dan collection.
- Mapping dari database/API ke model.

Yang harus bisa kamu jawab:

- Apa fungsi data class?
 - Kapan memakai enum?
 - Kapan memakai sealed class?
 - Bagaimana model API TomTom berbeda dari model internal aplikasi?
-

12. Repository Pattern

Repository menjadi penghubung antara UI/domain dengan sumber data.

Folder:

- `app/src/main/java/com/example/naikapa/data/repository/`

Repository penting:

- `TomTomSearchRepository`
- `TomTomRoutingRepository`
- `GtfsStopSearchRepository`
- `NearbyTransitStopRepository`
- `TransitGraphRepository`
- `TransitRoutingRepository`
- `CombinedRouteRepository`
- `RouteCacheRepository`
- `RouteCacheSerializer`

Contoh tanggung jawab:

- `TomTomSearchRepository`: mencari lokasi via TomTom Search API.
- `TomTomRoutingRepository`: menghitung rute motor/mobil via TomTom Routing API.
- `GtfsStopSearchRepository`: mencari halte/stasiun lokal dari database GTFS.
- `TransitRoutingRepository`: mencari rute transit dengan graph dan Dijkstra.
- `CombinedRouteRepository`: menggabungkan kendaraan pribadi + transit.
- `RouteCacheRepository`: mengambil dan menyimpan cache hasil rekomendasi rute.
- `RouteCacheSerializer`: mengubah `RecommendationResult` ke/dari JSON untuk disimpan di SQLite.

Yang harus dipahami:

- Fragment tidak seharusnya berisi semua query dan logika bisnis.
- Repository membuat kode lebih rapi dan mudah dites.
- Repository bisa mengambil data dari API, SQLite, atau memory.

Yang harus bisa kamu jawab:

- Kenapa `HomeFragment` tidak langsung memanggil Retrofit API?
 - Apa keuntungan repository untuk testing?
 - Apa bedanya DAO dan repository?
-

13. API, Retrofit, dan TomTom

Project ini memakai TomTom untuk:

- Pencarian lokasi.
- Rute kendaraan pribadi.

File penting:

- `data/remote/RemoteClient.kt`
- `data/remote/TomTomSearchApi.kt`
- `data/remote/TomTomRoutingApi.kt`
- `data/repository/TomTomSearchRepository.kt`
- `data/repository/TomTomRoutingRepository.kt`
- `data/model/TomTomSearchModels.kt`
- `data/model/TomTomRoutingModels.kt`

Konsep yang perlu dipahami:

- REST API.
- Base URL.
- Endpoint.
- Query parameter.
- API key.
- Retrofit interface.
- Gson JSON parsing.
- OkHttp logging.
- `Result<T>` untuk menangani sukses/gagal.

Pembagian file remote:

- `RemoteClient.kt` membuat satu konfigurasi Retrofit untuk base URL TomTom.
- `RemoteClient.kt` juga memasang `OkHttpClient` dan `HttpLoggingInterceptor` level BASIC.
- `TomTomSearchApi.kt` hanya mendefinisikan endpoint search lokasi.
- `TomTomRoutingApi.kt` hanya mendefinisikan endpoint routing asal-tujuan.
- Interface API tidak mengubah data ke model internal aplikasi; mapping dilakukan di repository.

Endpoint TomTom Search:

```
GET search/2/search/{query}.json
```

Parameter penting:

- `query`: teks pencarian user.
- `key`: API key TomTom.
- `countrySet = ID`: membatasi hasil ke Indonesia.
- `typeahead = true`: mendukung pencarian saat user mengetik.
- `limit`: jumlah hasil maksimum.
- `lat` dan `lon`: bias lokasi agar hasil dekat Jabodetabek/user lebih relevan.

Endpoint TomTom Routing:

```
GET routing/1/calculateRoute/{from}:{to}/json
```

Parameter penting:

- `from` dan `to`: koordinat asal/tujuan dalam format `lat, lon`.
- `travelMode`: `motorcycle`, `car`, atau `pedestrian`.
- `routeType = fastest`: meminta rute tercepat dari TomTom.
- `maxAlternatives`: jumlah alternatif rute.
- `avoid = tollRoads`: hanya dikirim untuk mobil saat user memilih hindari tol.

Alur search:

```
User mengetik tujuan
    |
    v
HomeFragment debounce input
    |
    v
TomTomSearchRepository
    |
    v
TomTomSearchApi
    |
```

```
      v
Response JSON
      |
      v
SearchLocation
```

Alur routing kendaraan pribadi:

```
Origin + destination + mode
      |
      v
TomTomRoutingRepository
      |
      v
TomTomRoutingApi
      |
      v
PrivateVehicleRouteResult
```

Alur routing jalan kaki TomTom:

```
Origin/destination walking segment
      |
      v
TomTomRoutingRepository.calculateWalkingRoute()
      |
      v
TomTomRoutingApi travelMode=pedestrian
      |
      v
WalkingRouteResult
```

Rute jalan kaki ini dipakai oleh `CombinedRouteRepository` untuk first-mile/last-mile jika API tersedia. Jika gagal, repository fallback ke jarak Haversine.

Estimasi biaya kendaraan pribadi:

```
jarakKm = distanceMeters / 1000
literTerpakai = jarakKm / konsumsiKmPerLiter
estimasiBbm = round(literTerpakai * hargaBbmPerLiter)
```

Di project ini, jarak kendaraan pribadi berasal dari TomTom Routing, bukan jarak garis lurus. `TomTomRoutingRepository` membaca `lengthInMeters` dan `travelTimeInSeconds` dari response TomTom, lalu `FuelCostCalculator` menghitung estimasi BBM.

Konstanta yang dipakai:

- `FUEL_COST_PER_LITER = 10000`
- `MOTOR_CONSUMPTION_KM_PER_LITER = 40.0`
- `CAR_CONSUMPTION_KM_PER_LITER = 12.0`

Contoh:

```
Motor 10 km = (10 / 40) * 10000 = Rp2.500
Mobil 10 km = (10 / 12) * 10000 = Rp8.333
```

Alasan rumus ini dipakai:

- Sederhana dan mudah dijelaskan.

- Cocok untuk membandingkan kendaraan pribadi dengan transportasi umum.
- Data jarak sudah tersedia dari TomTom Routing.
- Biaya BBM adalah komponen biaya kendaraan pribadi yang paling langsung dihitung dari jarak.

Catatan:

Estimasi ini belum memasukkan biaya tol, parkir, servis, depresiasi kendaraan, perubahan harga BBM, kondisi macet real-time, dan gaya berkendara. Jadi hasilnya adalah estimasi biaya BBM, bukan total biaya riil kendaraan.

Yang harus bisa kamu jawab:

- Apa itu Retrofit?
 - Kenapa API key disimpan di `local.properties`?
 - Apa yang terjadi jika internet mati?
 - Apa beda response API dan model internal aplikasi?
-

14. Coroutine dan Background Thread

Project ini banyak memakai coroutine karena ada pekerjaan berat:

- Request API.
- Query database.
- Hitung graph.
- Hitung rekomendasi.
- Simpan riwayat.

Konsep yang perlu dipelajari:

- `suspend fun`.
- `CoroutineScope`.
- `Dispatchers.Main`.
- `Dispatchers.IO`.
- `launch`.
- `async`.
- `await`.
- `withContext`.
- `SupervisorJob`.
- `Cancel coroutine`.

File contoh:

- `presentation/home/HomeFragment.kt`
- `domain/recommendation/RecommendationEngine.kt`
- `data/repository/TomTomSearchRepository.kt`
- `data/repository/TomTomRoutingRepository.kt`
- `data/repository/CombinedRouteRepository.kt`

Contoh penting:

- Search input memakai debounce supaya API tidak dipanggil setiap huruf secara berlebihan.
- Recommendation engine menjalankan beberapa kandidat rute secara paralel.
- Save history dilakukan di `Dispatchers.IO`.

Yang harus bisa kamu jawab:

- Kenapa network/database tidak boleh jalan di main thread?
 - Apa beda `launch` dan `async`?
 - Kenapa coroutine perlu dibatalkan di `onDestroyView()`?
-

15. Peta dan Lokasi

Project ini memakai:

- `osmdroid` untuk peta `OpenStreetMap`.
- `Google Play Services Location` untuk lokasi perangkat.
- `CartoDB tiles` untuk tampilan peta.

File penting:

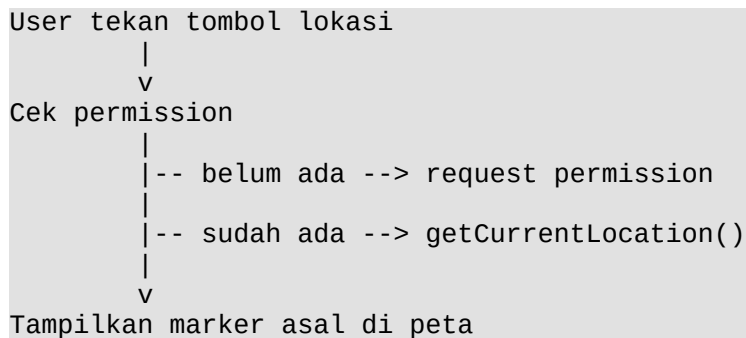
- `presentation/home/HomeFragment.kt`
- `data/model/MapModels.kt`
- `common/AppConstants.kt`

Konsep yang perlu dipahami:

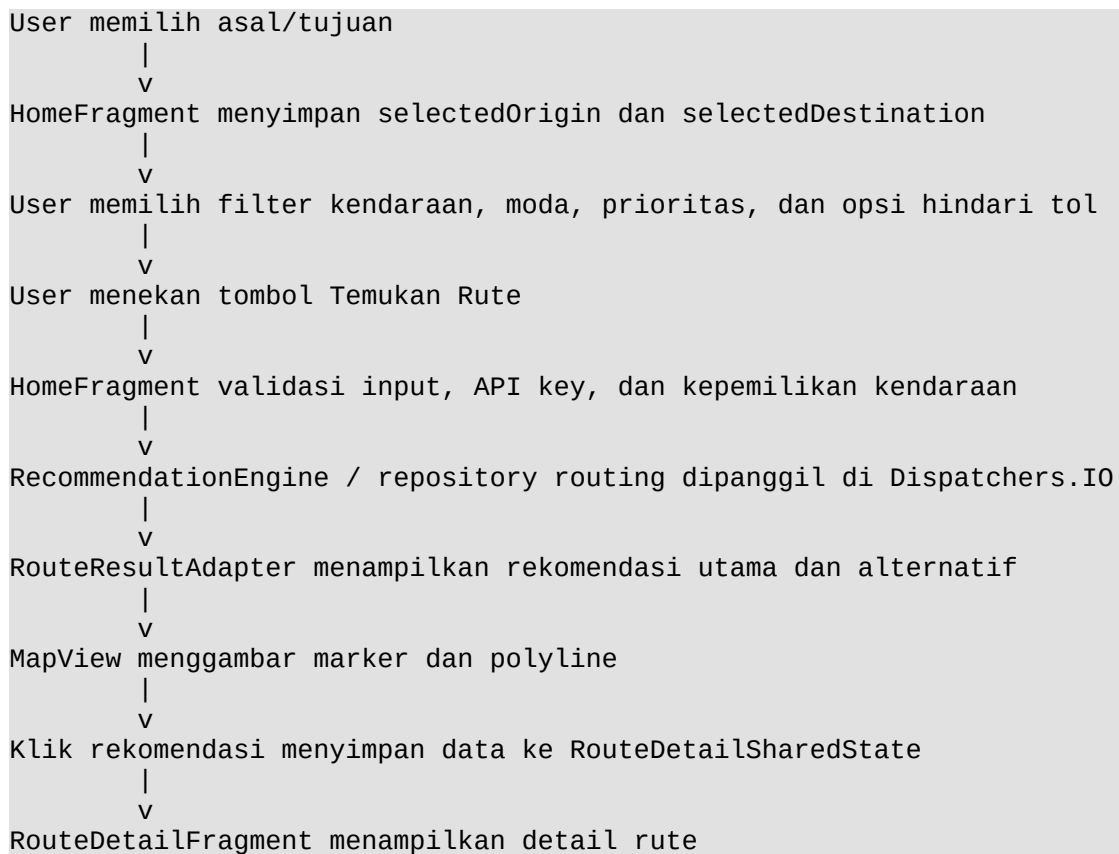
- Runtime permission lokasi.
- `FusedLocationProviderClient`.
- `MapView`.
- Tile source.
- Marker.
- Polyline.

- Zoom dan center map.
- Lifecycle map: `onResume()` dan `onPause()`.

Alur lokasi:



Alur lengkap HomeFragment:



Detail rendering peta:

- HomeFragment menginisialisasi MapView dengan osmdroid dan tile CartoDB.
- Tersedia style peta POSITRON dan DARK_MATTER.
- Marker asal memakai `MapMarkerType.ORIGIN` dan icon lokasi.
- Marker tujuan memakai `MapMarkerType.DESTINATION` dan icon tujuan.
- Marker transit memakai `MapMarkerType.TRANSIT` dan icon kereta.
- Polyline motor memakai `colorMotor`.

- Polyline mobil memakai `colorMobil`.
- Polyline jalan kaki memakai `colorWalking`.
- Polyline transit memakai warna rute dari GTFS jika tersedia, lalu fallback ke warna agency seperti TransJakarta/KRL/MRT/LRT.
- `RouteDetailFragment` menggambar ulang marker dan polyline di peta detail.
- Mode fullscreen menyalin overlay dari peta detail ke `fullscreenMapView`.
- `TransitMarkerRole.BOARD` dipakai untuk titik naik, sedangkan `TransitMarkerRole.ALIGHT` untuk titik turun.

Yang harus bisa kamu jawab:

- Kenapa permission lokasi harus diminta saat runtime?
 - Apa beda `ACCESS_FINE_LOCATION` dan `ACCESS_COARSE_LOCATION`?
 - Apa fungsi marker dan polyline?
 - Kenapa `MapView` perlu `onResume()` dan `onPause()`?
-

16. Algoritma Routing Transit

Routing transit adalah bagian domain utama aplikasi.

Folder:

- `app/src/main/java/com/example/naikapa/domain/routing/`

File penting:

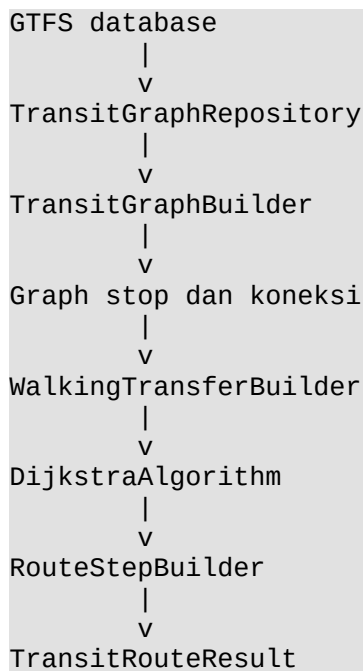
- `TransitGraphBuilder.kt`
- `WalkingTransferBuilder.kt`
- `DijkstraAlgorithm.kt`
- `TransitEdgeCostCalculator.kt`
- `RouteStepBuilder.kt`
- `TransitRouteMetricsCalculator.kt`
- `FareCalculator.kt`
- `GeoDistanceCalculator.kt`
- `GtfsTimeParser.kt`

Konsep yang perlu dipahami:

- Graph.
- Node.
- Edge.

- Weight/cost.
- Dijkstra algorithm.
- Walking transfer.
- Fare calculation.
- Route step/timeline.
- Metrik rute: durasi, biaya, jalan kaki, jumlah transit.

Alur routing transit:



Base algoritma Dijkstra di project ini:

node = halte/stasiun
 edge = koneksi antar stop atau transfer jalan kaki
 weight/cost = nilai yang dihitung dari durasi, biaya, jalan kaki, atau transfer

Graph dibangun dari data GTFS lokal:

1. `GtfsDao.getAdjacentStopConnections()` mengambil pasangan stop berurutan dari tabel GTFS.
2. `TransitGraphBuilder` membuat node dari stop dan edge dari koneksi antar stop.
3. Durasi edge transit dihitung dari selisih `departure_time` dan `arrival_time`, lalu dirata-ratakan untuk koneksi yang sama.
4. Jarak edge dihitung dari koordinat stop dengan `GeoDistanceCalculator`.
5. `WalkingTransferBuilder` menambahkan edge jalan kaki antar stop beda agency yang jaraknya masih dalam radius transfer.

Implementasi Dijkstra:

1. `TransitRoutingRepository.findRoute()` mengambil graph dari

TransitGraphRepository.

2. `DijkstraAlgorithm.findPath()` menerima graph, stop asal, stop tujuan, mode transit, dan preferensi sort.
3. Algoritma menyimpan cost terkecil sementara di `distances`.
4. `PriorityQueue` selalu mengambil stop dengan cost paling kecil.
5. Setiap edge keluar dari stop tersebut dicek apakah boleh dipakai sesuai `TransitMode`.
6. `TransitEdgeCostCalculator` menghitung cost edge sesuai preferensi user.
7. Jika cost baru lebih kecil, `distances` dan `previous` di-update.
8. Setelah tujuan ditemukan, path direkonstruksi dari `previous`.
9. `RouteStepBuilder` mengubah path edge menjadi langkah perjalanan yang bisa dibaca user.
10. `TransitRouteMetricsCalculator` menghitung total durasi, jarak, biaya, jalan kaki, dan jumlah transit.

Cost yang dipakai Dijkstra berubah mengikuti preferensi:

FASTEST	= durasi edge + penalti transfer
CHEAPEST	= biaya masuk agency + biaya transfer + tie-breaker durasi
MIN_WALKING	= jarak jalan kaki * bobot + sebagian penalti transfer
FEWEST_TRANSFERS	= jumlah transfer + tie-breaker durasi

Konstanta penting:

- `ROUTING_TRANSFER_PENALTY_SECONDS` = 300
- `ROUTING_AGENCY_ENTRY_COST` = 3500
- `ROUTING_TRANSFER_COST` = 500
- `ROUTING_WALKING_DISTANCE_WEIGHT` = 10.0
- `ROUTING_DURATION_TIEBREAKER` = 0.00001

Rumus tarif transportasi umum:

`totalFare` = jumlah tarif per agency yang dipakai dalam route step

Detail per operator:

- TransJakarta (Tij): tarif flat `FARE_TRANSJAKARTA_FLAT` = 3500.
- KRL (KAIC): `FARE_KRL_BASE` = 3000 untuk jarak sampai 25 km, lalu tambah 1000 setiap blok 10 km berikutnya.
- MRT (MRTJ): `FARE_MRT_BASE` = 3000 + jumlah_stop_dilewati * 1000, dibatasi maksimum `FARE_MRT_MAX` = 14000.
- LRT Jakarta (LRTJ): tarif flat `FARE_LRTJ_FLAT` = 5000.
- LRT Jabodebek (LRTJB): `FARE_LRTJB_BASE` = 5000 untuk 1 km pertama, lalu

tambah sekitar 700/km, dibatasi maksimum $FARE_LRTJB_MAX = 20000$.

- Walking step selalu bernilai 0.

Contoh KRL:

```
Jarak 30 km  
base 25 km = Rp3.000  
extra 5 km masuk 1 blok 10 km = Rp1.000  
total = Rp4.000
```

Keterbatasan algoritma transit saat ini:

- Belum time-dependent routing penuh; Dijkstra tidak memilih rute berdasarkan jam keberangkatan aktual user.
- Edge transit memakai rata-rata durasi dari koneksi GTFS yang sama, bukan jadwal real-time.
- Headway/frekuensi kedatangan kendaraan belum dihitung detail sebagai waktu tunggu.
- Walking transfer hanya dibuat berdasarkan radius antar stop dan beda agency.
- Gangguan aktif memengaruhi scoring rekomendasi, tetapi belum menghapus edge dari graph secara langsung.

Kenapa memilih Dijkstra:

- Cocok untuk graph berbobot non-negatif, dan semua cost routing di project ini bernilai nol atau positif.
- Lebih tepat daripada BFS, karena BFS menganggap semua edge setara, sedangkan rute transit punya durasi, biaya, jarak jalan kaki, dan penalti transfer yang berbeda.
- Lebih sederhana daripada A*, karena A* butuh heuristic yang benar. Untuk transit multi-moda, heuristic jarak geografis belum tentu cocok untuk preferensi biaya, minim jalan kaki, atau minim transfer.
- Lebih efisien daripada Bellman-Ford untuk kasus ini, karena tidak ada edge berbobot negatif.
- Lebih relevan daripada Floyd-Warshall, karena aplikasi mencari rute asal-tujuan saat user melakukan pencarian, bukan menghitung semua pasangan stop.

Yang harus bisa kamu jawab:

- Apa itu graph dalam konteks transportasi?
 - Apa yang menjadi node dan edge?
 - Kenapa perlu walking transfer?
 - Bagaimana Dijkstra memilih rute?
 - Bagaimana biaya dihitung untuk KRL, MRT, LRT, dan TransJakarta?
-

17. Rute Gabungan

Rute gabungan berarti kendaraan pribadi + transportasi umum.

Contoh:

```
Motor dari rumah ke stasiun
    |
    v
KRL/MRT/LRT/TransJakarta
    |
    v
Jalan kaki ke tujuan
```

File penting:

- data/repository/CombinedRouteRepository.kt
- data/repository/NearbyTransitStopRepository.kt
- data/repository/TomTomRoutingRepository.kt
- data/repository/TransitRoutingRepository.kt
- data/model/CombinedRouteModels.kt

Konsep yang perlu dipahami:

- Mencari stop terdekat dari asal.
- Mencari stop terdekat dari tujuan.
- Menghitung rute kendaraan pribadi ke stop.
- Menghitung rute transit antar stop.
- Menggabungkan beberapa segmen.
- Menghitung total metrik.

Alur internal CombinedRouteRepository:

```
origin/destination koordinat
    |
    v
NearbyTransitStopRepository mencari kandidat stop asal dan tujuan
    |
    v
Kombinasi stop dibatasi oleh COMBINED_ROUTE_MAX_COMBINATIONS
    |
    v
Jika pakai motor/mobil, TomTom menghitung rute kendaraan ke stop asal
    |
    v
TransitRoutingRepository menghitung rute transit antar stop
    |
    v
TomTom pedestrian menghitung first-mile/last-mile jalan kaki jika diperlukan
    |
    v
Jika TomTom pedestrian gagal, fallback ke Haversine
    |
    v
```

Segmen digabung menjadi CombinedRouteResult

Kenapa kombinasi stop dibatasi:

- Stop terdekat dari asal dan tujuan bisa banyak.
- Mencoba semua kombinasi akan membuat API call dan Dijkstra terlalu banyak.
- Project memakai limit kandidat dan COMBINED_ROUTE_MAX_COMBINATIONS agar pencarian tetap responsif.

Biaya total rute gabungan:

$\text{estimatedTotalCost} = \text{estimatedFare} + \text{estimatedBbm}$

Artinya:

- `estimatedFare` berasal dari tarif transportasi umum.
- `estimatedBbm` berasal dari segmen kendaraan pribadi ke halte/stasiun awal.
- Jika rute gabungan tidak memakai kendaraan pribadi, `estimatedBbm = 0`.

Yang harus bisa kamu jawab:

- Kenapa tidak semua kombinasi stop dihitung?
- Apa fungsi radius pencarian stop?
- Bagaimana total durasi dan biaya digabung?

18. Sistem Rekomendasi dan Scoring

Ini bagian yang menentukan rute mana yang dianggap paling cocok.

Folder:

- `app/src/main/java/com/example/naikapa/domain/recommendation/`

File penting:

- `RecommendationEngine.kt`
- `RecommendationScorer.kt`
- `RecommendationReasonBuilder.kt`
- `data/model/RecommendationModels.kt`
- `common/AppConstants.kt`

Alur rekomendasi:

```
Input user:
origin, destination, mode, priority, kendaraan user
    |
    v
RecommendationEngine
    |
```

```

      v
Hitung kandidat:
transit, motor, mobil, gabungan
      |
      v
Ambil gangguan aktif
      |
      v
RecommendationScorer
      |
      v
Urutkan skor
      |
      v
Rekomendasi utama + alternatif

```

Prioritas scoring:

- Tercepat.
- Terhemat.
- Minim jalan kaki.
- Minim transit.

Dimensi scoring:

- Waktu.
- Biaya.
- Jarak jalan kaki.
- Jumlah transit.
- Gangguan aktif.

Rumus scoring rekomendasi:

```

nilai_dimensi = 1 - (nilai_aktual / nilai_referensi_maksimum)
nilai_dimensi dibatasi 0.0 sampai 1.0

rawScore =
    timeWeight      * timeScore +
    costWeight      * costScore +
    walkingWeight   * walkingScore +
    transitWeight   * transitScore

finalScore = clamp(round(rawScore) - disruptionPenalty, 0, 100)

```

Referensi normalisasi:

- Durasi maksimum referensi: 7200 detik atau 2 jam.
- Biaya maksimum referensi: Rp50.000.
- Jalan kaki maksimum referensi: 3000 meter.
- Jumlah transit maksimum referensi: 5 kali.

Bobot per prioritas:

Prioritas	Waktu	Biaya	Jalan Kaki	Transit
Tercepat	45	15	15	15
Terhemat	20	45	15	10
Minim Jalan Kaki	20	15	45	10
Minim Transit	20	15	10	45

Penalti gangguan:

- Jika kandidat transit/gabungan melewati stop atau route yang punya laporan gangguan aktif, skor dikurangi `SCORE_DISRUPTION_PENALTY = 15`.
- Kandidat kendaraan pribadi tidak dianggap terdampak gangguan transit.
- Catatan implementasi saat ini: konstanta `SCORE_W_*_DISRUPTION` tersedia di `AppConstants`, tetapi `RecommendationScorer` memakai penalti tetap 15, bukan bobot disruption sebagai dimensi terpisah.

Kenapa rute lambat bisa menang:

- Scoring memakai banyak dimensi, bukan hanya durasi.
- Jika user memilih **Terhemat**, bobot biaya lebih besar daripada waktu.
- Jika user memilih **Minim Jalan Kaki**, rute yang lebih lama bisa menang jika jarak jalan kakinya jauh lebih kecil.
- Jika ada gangguan aktif, rute yang sebenarnya bagus bisa turun peringkat karena penalti gangguan.

Yang harus bisa kamu jawab:

- Kenapa hasil rekomendasi tidak selalu rute tercepat?
- Bagaimana prioritas user memengaruhi skor?
- Apa efek laporan gangguan terhadap skor?
- Kenapa perlu alasan rekomendasi, bukan hanya angka skor?

19. Riwayat, Favorit, dan Replay Perjalanan

Fitur ini menyimpan aktivitas user agar bisa dilihat lagi.

File penting:

- `presentation/history/RiwayatFragment.kt`
- `presentation/history/SearchHistoryAdapter.kt`
- `presentation/history/RouteHistoryAdapter.kt`
- `presentation/history/SavedTripAdapter.kt`

- `presentation/history/HistoryReplayRequest.kt`
- `data/local/HistoryDao.kt`
- `data/local/SavedTripDao.kt`
- `data/model/SearchHistory.kt`
- `data/model/RouteHistory.kt`
- `data/model/SavedTrip.kt`

Tabel terkait:

- `search_history`
- `route_history`
- `saved_trips`

Konsep yang perlu dipahami:

- Menyimpan riwayat pencarian.
- Menyimpan riwayat rekomendasi rute.
- Menyimpan perjalanan favorit.
- Adapter RecyclerView untuk menampilkan list.
- Replay request dari riwayat ke halaman home.

Yang harus bisa kamu jawab:

- Kapan riwayat disimpan?
- Bagaimana data riwayat ditampilkan di RecyclerView?
- Bagaimana klik riwayat bisa mengisi ulang origin/destination?

20. Laporan Gangguan

Fitur laporan gangguan memungkinkan user membuat laporan yang aktif selama periode tertentu.

File penting:

- `presentation/report/StatusGangguanFragment.kt`
- `presentation/report/AddEditDisruptionReportFragment.kt`
- `presentation/report/DisruptionReportAdapter.kt`
- `presentation/report/ReportPhotoHelper.kt`
- `data/local/DisruptionReportDao.kt`
- `data/model/DisruptionReport.kt`

Tabel terkait:

- `disruption_reports`

Konsep yang perlu dipahami:

- CRUD laporan gangguan.
- Ambil foto dari kamera.
- Ambil foto dari galeri.
- FileProvider.
- Status laporan aktif/expired.
- Laporan gangguan memengaruhi scoring rekomendasi.

Alur foto laporan dari kamera:

```
User tekan tombol kamera
    |
    v
Cek permission CAMERA
    |
    v
ReportPhotoHelper.createPhotoFile()
    |
    v
File dibuat di cache/disruption_photos
    |
    v
ReportPhotoHelper.getUriForFile()
    |
    v
ActivityResultContracts.TakePicture menyimpan foto ke Uri
    |
    v
Path file dinormalisasi dan disimpan sebagai photo_path
```

Alur foto laporan dari galeri:

```
User tekan tombol galeri
    |
    v
Cek READ_MEDIA_IMAGES untuk Android 13+ atau READ_EXTERNAL_STORAGE untuk Android
12 ke bawah
    |
    v
ActivityResultContracts.GetContent memilih image/*
    |
    v
File dari content resolver disalin ke cache/disruption_photos
    |
    v
Path file lokal disimpan sebagai photo_path
```

Alur tampil dan hapus foto:

- Saat edit laporan, photo_path dibaca dari SQLite.
- Jika file masih ada, preview ditampilkan dengan `Uri.fromFile(file)`.
- Saat laporan dihapus, file foto ikut dihapus melalui `ReportPhotoHelper.deletePhoto`.

- Tombol hapus foto di form menghapus pilihan foto dari form, tetapi penghapusan file permanen dilakukan saat data laporan dihapus.

Yang harus bisa kamu jawab:

- Bagaimana laporan dibuat?
 - Bagaimana foto laporan disimpan?
 - Kenapa laporan punya `expired_at`?
 - Bagaimana gangguan aktif memengaruhi rekomendasi?
-

21. Profil User

Profil menyimpan informasi dan preferensi user.

File penting:

- `presentation/profile/ProfilFragment.kt`
- `data/local/UserDao.kt`
- `data/model/User.kt`
- `data/model/UserProfile.kt`
- `common/SessionManager.kt`

Data profil:

- Nama.
- Email.
- Kepemilikan motor.
- Kepemilikan mobil.
- Preferensi moda.
- Preferensi prioritas.

Yang harus bisa kamu jawab:

- Bagaimana data profil dibaca dari database?
 - Bagaimana update profil memengaruhi session?
 - Bagaimana kepemilikan kendaraan memengaruhi rekomendasi?
-

22. Testing

Project ini sudah memiliki unit test dan instrumented test.

Folder:

- `app/src/test/`
- `app/src/androidTest/`

Contoh unit test:

- `DijkstraAlgorithmTest.kt`
- `TransitGraphBuilderTest.kt`
- `WalkingTransferBuilderTest.kt`
- `RecommendationScorerTest.kt`
- `FareCalculatorTest.kt`
- `FuelCostCalculatorTest.kt`
- `TomTomRoutingRepositoryTest.kt`
- `CombinedRouteRepositoryTest.kt`
- `NaikApaDatabaseHelperTest.kt`

Contoh instrumented test:

- `NaikApaDatabaseInstrumentedTest.kt`
- `FavoriteHistoryDaoInstrumentedTest.kt`

Perintah penting:

```
./gradlew testDebugUnitTest
./gradlew connectedAndroidTest
./gradlew lintDebug
./gradlew assembleDebug
```

Di Windows PowerShell, biasanya bisa memakai:

```
.\gradlew.bat testDebugUnitTest
.\gradlew.bat lintDebug
.\gradlew.bat assembleDebug
```

Yang harus bisa kamu jawab:

- Apa beda unit test dan instrumented test?
- Bagian mana yang sebaiknya dites dengan unit test?
- Bagian mana yang butuh emulator/device?
- Kenapa algoritma routing perlu test?

23. Cara Membuat Project Seperti Ini dari Android Studio

Urutan besar jika membuat dari awal:

1. Buat project Android baru.
2. Pilih Kotlin.

3. Pakai Empty Views Activity jika ingin XML layout.
4. Atur package name.
5. Tambahkan dependency di `app/build.gradle.kts`.
6. Aktifkan ViewBinding.
7. Buat struktur package:
 - `common`
 - `data/local`
 - `data/model`
 - `data/remote`
 - `data/repository`
 - `domain/routing`
 - `domain/recommendation`
 - `presentation`
8. Buat layout XML.
9. Buat Activity dan Fragment.
10. Tambahkan Navigation Component.
11. Buat database contract dan helper.
12. Buat DAO.
13. Buat model.
14. Buat repository.
15. Tambahkan API Retrofit.
16. Tambahkan peta dan permission lokasi.
17. Tambahkan algoritma routing.
18. Tambahkan sistem rekomendasi.
19. Tambahkan fitur riwayat, favorit, profil, laporan.
20. Tambahkan test.

Dependency utama yang perlu dipahami:

- AndroidX Core KTX.
- AppCompat.
- Material Components.
- ConstraintLayout.
- Navigation Component.

- Retrofit.
 - OkHttp.
 - Gson converter.
 - Coroutines.
 - osmdroid.
 - Google Play Services Location.
 - Glide.
 - JUnit.
 - Espresso.
 - Mockito.
-

24. Urutan Praktik Membaca Kode

Gunakan urutan ini saat belajar langsung dari source code.

Hari 1: Struktur dan Build

Baca:

- `README.md`
- `settings.gradle.kts`
- `app/build.gradle.kts`
- `AndroidManifest.xml`

Praktik:

- Buka project di Android Studio.
- Sync Gradle.
- Run app di emulator/device.
- Cek Logcat.

Hari 2: UI dan Navigasi

Baca:

- `activity_splash.xml`
- `activity_auth.xml`
- `activity_main.xml`
- `fragment_login.xml`
- `fragment_register.xml`

- `nav_auth.xml`
- `nav_main.xml`
- `SplashActivity.kt`
- `AuthActivity.kt`
- `MainActivity.kt`
- `LoginFragment.kt`
- `RegisterFragment.kt`

Praktik:

- Ubah teks kecil di UI.
- Tambahkan satu Toast.
- Ikuti flow login-register.

Hari 3: Session dan Database User

Baca:

- `SessionManager.kt`
- `NaikApaDbContract.kt`
- `NaikApaDatabaseHelper.kt`
- `UserDao.kt`
- `User.kt`
- `UserProfile.kt`

Praktik:

- Telusuri proses register.
- Telusuri proses login.
- Cek data apa yang disimpan di session.

Hari 4: Home, Search, Lokasi, dan Peta

Baca:

- `HomeFragment.kt`
- `fragment_home.xml`
- `MapModels.kt`
- `LocationPoint.kt`
- `SearchLocation.kt`
- `TomTomSearchRepository.kt`

- `GtfsStopSearchRepository.kt`

Praktik:

- Jalankan fitur lokasi.
- Coba search tujuan.
- Lihat marker di peta.

Hari 5: API TomTom

Baca:

- `RemoteClient.kt`
- `TomTomSearchApi.kt`
- `TomTomRoutingApi.kt`
- `TomTomSearchModels.kt`
- `TomTomRoutingModels.kt`
- `TomTomRoutingRepository.kt`

Praktik:

- Pastikan `TOMTOM_API_KEY` tersedia.
- Jalankan search dan routing.
- Lihat response di Logcat jika logging aktif.

Hari 6: GTFS dan Routing Transit

Baca:

- `PrebuiltDatabaseCopier.kt`
- `GtfsDao.kt`
- `TransitGraphRepository.kt`
- `TransitGraphBuilder.kt`
- `WalkingTransferBuilder.kt`
- `DijkstraAlgorithm.kt`
- `TransitRoutingRepository.kt`

Praktik:

- Cari stop di database.
- Telusuri cara graph dibangun.
- Baca unit test Dijkstra.

Hari 7: Rekomendasi

Baca:

- `RecommendationEngine.kt`
- `RecommendationScorer.kt`
- `RecommendationReasonBuilder.kt`
- `RecommendationModels.kt`
- `CombinedRouteRepository.kt`

Praktik:

- Coba beberapa prioritas berbeda.
- Bandingkan perubahan skor.
- Telusuri kenapa rekomendasi utama berubah.

Hari 8: Riwayat, Favorit, dan Laporan

Baca:

- `RiwayatFragment.kt`
- `HistoryDao.kt`
- `SavedTripDao.kt`
- `StatusGangguanFragment.kt`
- `AddEditDisruptionReportFragment.kt`
- `DisruptionReportDao.kt`

Praktik:

- Simpan rute.
- Lihat riwayat.
- Buat laporan gangguan.
- Cek apakah gangguan berpengaruh ke rekomendasi.

Hari 9: Testing

Baca:

- `DijkstraAlgorithmTest.kt`
- `RecommendationScorerTest.kt`
- `FareCalculatorTest.kt`
- `NaikApaDatabaseHelperTest.kt`

Praktik:

```
.\gradlew.bat testDebugUnitTest
```

```
.\gradlew.bat lintDebug
.\gradlew.bat assembleDebug
```

25. Checklist Kompetensi

Gunakan checklist ini untuk mengukur pemahaman.

Dasar Android

- ☐ Bisa menjelaskan fungsi Activity.
- ☐ Bisa menjelaskan fungsi Fragment.
- ☐ Bisa menjelaskan lifecycle dasar.
- ☐ Bisa membaca XML layout.
- ☐ Bisa memakai ViewBinding.
- ☐ Bisa memahami Navigation Component.
- ☐ Bisa membaca `AndroidManifest.xml`.

Kotlin

- ☐ Bisa membaca data class.
- ☐ Bisa membaca enum class.
- ☐ Bisa membaca sealed class.
- ☐ Bisa memahami nullable type.
- ☐ Bisa memahami lambda/list/map/filter.
- ☐ Bisa memahami coroutine dasar.

Database

- ☐ Bisa menjelaskan SQLiteOpenHelper.
- ☐ Bisa membaca SQL `CREATE TABLE`.
- ☐ Bisa memahami foreign key.
- ☐ Bisa memahami index.
- ☐ Bisa membaca DAO.
- ☐ Bisa mapping Cursor ke model.

Networking

- ☐ Bisa menjelaskan Retrofit.
- ☐ Bisa membaca interface API.
- ☐ Bisa memahami query parameter.

- ☐ Bisa memahami model response JSON.
- ☐ Bisa menangani error API.

Project NaikApa

- ☐ Bisa menjelaskan alur splash-login-home.
 - ☐ Bisa menjelaskan alur pencarian lokasi.
 - ☐ Bisa menjelaskan alur lokasi GPS.
 - ☐ Bisa menjelaskan alur peta dan marker.
 - ☐ Bisa menjelaskan alur GTFS database.
 - ☐ Bisa menjelaskan graph transit.
 - ☐ Bisa menjelaskan Dijkstra.
 - ☐ Bisa menjelaskan rute gabungan.
 - ☐ Bisa menjelaskan scoring rekomendasi.
 - ☐ Bisa menjelaskan riwayat dan favorit.
 - ☐ Bisa menjelaskan laporan gangguan.
-

26. Pertanyaan Latihan

Jawab pertanyaan ini setelah belajar.

1. Kenapa project ini memakai `SplashActivity`?
2. Apa yang dilakukan `SessionManager`?
3. Kenapa `HomeFragment` menjadi file yang besar?
4. Apa beda `TomTomSearchRepository` dan `TomTomRoutingRepository`?
5. Apa beda `GtfsDao` dan `TransitRoutingRepository`?
6. Kenapa database GTFS disimpan sebagai asset?
7. Apa fungsi `PrebuiltDatabaseCopier`?
8. Apa itu stop, route, trip, dan stop time dalam GTFS?
9. Bagaimana graph transportasi dibangun?
10. Bagaimana walking transfer dibuat?
11. Bagaimana Dijkstra mencari rute?
12. Bagaimana sistem menghitung biaya perjalanan?
13. Apa itu rute gabungan?
14. Bagaimana prioritas user memengaruhi skor?

15. Bagaimana laporan gangguan memengaruhi rekomendasi?
 16. Kenapa operasi API dan database perlu coroutine?
 17. Kenapa MapView perlu lifecycle handling?
 18. Apa beda unit test dan instrumented test?
 19. Bagaimana cara menambah fitur baru dengan mengikuti arsitektur project?
 20. Bagian mana dari project ini yang paling berisiko bug dan perlu test?
-

27. Tips Belajar Efektif

- Jangan mulai dari HomeFragment secara penuh. File itu besar, jadi baca per fungsi.
 - Baca dari alur user, bukan dari urutan file.
 - Untuk setiap fitur, cari tiga lapisan: UI, repository/DAO, model/domain.
 - Gunakan fitur "Find Usages" di Android Studio.
 - Gunakan breakpoint dan debugger.
 - Gunakan Logcat untuk melihat alur runtime.
 - Saat bingung, cari dulu model data yang dipakai.
 - Saat mempelajari algoritma, baca unit test-nya karena biasanya lebih kecil dan jelas.
-

28. Jalur Cepat Kalau Waktu Terbatas

Kalau hanya punya sedikit waktu untuk presentasi atau memahami project secara umum, fokus ke ini:

1. README.md
2. AndroidManifest.xml
3. SplashActivity.kt
4. MainActivity.kt
5. LoginFragment.kt
6. HomeFragment.kt, khusus bagian:
 - setup repository
 - search lokasi
 - lokasi GPS
 - tombol cari rute
 - tampilkan rekomendasi

7. `NaikApaDbContract.kt`
8. `NaikApaDatabaseHelper.kt`
9. `GtfsDao.kt`
10. `TransitRoutingRepository.kt`
11. `DijkstraAlgorithm.kt`
12. `RecommendationEngine.kt`
13. `RecommendationScorer.kt`

Dengan membaca file-file itu, kamu sudah bisa menjelaskan inti aplikasi:

- Bagaimana user masuk aplikasi.
 - Bagaimana user memilih asal dan tujuan.
 - Bagaimana data lokasi dicari.
 - Bagaimana rute dihitung.
 - Bagaimana rekomendasi dipilih.
 - Bagaimana data disimpan.
-

29. Penjelasan File yang Disebut di Roadmap

Bagian ini adalah katalog penjelasan file. Gunakan bagian ini saat kamu menemukan nama file di roadmap dan ingin tahu file itu untuk apa, posisinya di arsitektur, serta cara kerjanya secara umum.

Dokumentasi dan Konsep

README.md

File ini adalah pintu masuk utama untuk memahami project.

Isinya menjelaskan:

- Nama dan tujuan aplikasi.
- Fitur utama.
- Arsitektur umum.
- Database.
- Tech stack.
- Cara menjalankan aplikasi.
- Data transit yang digunakan.
- Alur pengguna utama.

Cara kerjanya:

- File ini tidak dijalankan oleh aplikasi.
- Fungsinya sebagai dokumentasi untuk developer, dosen, reviewer, atau anggota tim.
- Saat pertama kali membuka project, baca file ini sebelum membaca kode Kotlin.

konsep/prd_naik_apa_mvp.md

File ini berisi PRD atau Product Requirement Document untuk versi MVP.

Isinya biasanya menjelaskan:

- Masalah yang ingin diselesaikan.
- Target pengguna.
- Fitur yang wajib ada.
- Fitur yang bisa ditunda.
- Batasan scope project.

Cara kerjanya:

- File ini menjadi acuan kenapa fitur tertentu dibuat.
- Saat kamu bingung "kenapa aplikasi punya fitur ini", cek PRD.
- PRD membantu membedakan fitur inti dan fitur tambahan.

konsep/KONSEP_TEKNIS.md

File ini menjelaskan rancangan teknis aplikasi.

Isinya biasanya membahas:

- Arsitektur aplikasi.
- Alur data.
- Cara database digunakan.
- Cara routing/rekomendasi bekerja.
- Keputusan teknis yang dipilih.

Cara kerjanya:

- File ini menjadi jembatan antara ide produk dan implementasi kode.
- Cocok dibaca sebelum masuk ke file domain seperti `DijkstraAlgorithm.kt` atau `RecommendationEngine.kt`.

konsep/NaikApa_PLAN.md

File ini berisi rencana implementasi atau catatan pengembangan.

Cara kerjanya:

- Biasanya dipakai untuk melacak pekerjaan yang sudah/akan dikerjakan.
- Bisa menjadi referensi urutan pembuatan fitur.

konsep/Prd Naik Apa Mvp.pdf

Versi PDF dari dokumen PRD.

Cara kerjanya:

- Sama seperti PRD markdown, tetapi formatnya siap dibaca/dibagikan.
- Tidak dipakai langsung oleh aplikasi.

konsep/NaikApa_Konsep_Aplikasi_Revisi_Inovasi.pdf

Dokumen konsep aplikasi dalam bentuk PDF.

Cara kerjanya:

- Dipakai sebagai referensi konseptual.
 - Tidak memengaruhi build Android secara langsung.
-

File Build dan Konfigurasi Project

settings.gradle.kts

File ini mendefinisikan konfigurasi root project Gradle.

Isinya penting karena:

- Menentukan nama root project.
- Menentukan modul yang ikut dibuild, yaitu `include(":app")`.
- Mengatur repository plugin dan dependency.

Cara kerjanya:

- Saat Android Studio melakukan Gradle Sync, file ini dibaca dulu.
- Gradle memakai file ini untuk tahu module mana saja yang ada.
- Di project ini, hanya ada satu module aplikasi: `app`.

build.gradle.kts

Ini adalah build script tingkat root project.

Isinya:

- Plugin Android Application disediakan untuk subproject.
- Konfigurasi global yang bisa dipakai module.

Cara kerjanya:

- File ini bukan tempat utama dependency aplikasi.

- Dependency aplikasi berada di `app/build.gradle.kts`.
- Root build file lebih sering dipakai untuk konfigurasi global.

app/build.gradle.kts

Ini adalah build script utama untuk aplikasi Android.

Isinya:

- `namespace`.
- `applicationId`.
- `compileSdk`, `minSdk`, `targetSdk`.
- Version code dan version name.
- Test runner.
- Pengambilan `TOMTOM_API_KEY` dari `local.properties`.
- Build type debug/release.
- Aktivasi `ViewBinding` dan `BuildConfig`.
- Daftar dependency aplikasi.

Cara kerjanya:

- Saat build, Gradle membaca file ini untuk tahu cara mengompilasi app.
- Dependency seperti `Retrofit`, `osmdroid`, `Material`, `Navigation`, dan `Coroutines` ditarik dari sini.
- API key TomTom dimasukkan ke `BuildConfig.TOMTOM_API_KEY`, lalu bisa dipakai dari Kotlin.

gradle/libs.versions.toml

File ini menyimpan daftar versi plugin dan library.

Cara kerjanya:

- Dependency di `app/build.gradle.kts` bisa ditulis sebagai `libs.retrofit`, `libs.material`, dan sejenisnya.
- Versi aslinya disimpan di file ini supaya lebih rapi dan terpusat.
- Jika ingin upgrade library, biasanya cek dan ubah versi di sini.

gradle.properties

File konfigurasi Gradle.

Cara kerjanya:

- Bisa berisi konfigurasi performa Gradle.
- Bisa juga berisi property tambahan.

- Jangan menaruh secret yang seharusnya lokal jika file ini ikut di-commit.

local.properties

File lokal untuk konfigurasi komputer developer.

Di project ini dipakai untuk:

- Lokasi Android SDK.
- TOMTOM_API_KEY.

Cara kerjanya:

- File ini dibaca oleh `app/build.gradle.kts`.
 - Nilai TOMTOM_API_KEY diubah menjadi `BuildConfig.TOMTOM_API_KEY`.
 - File ini biasanya tidak di-commit karena berisi konfigurasi lokal/secret.
-

Manifest dan Resource Umum

app/src/main/AndroidManifest.xml

Manifest adalah deklarasi utama aplikasi Android.

Isinya:

- Permission internet, lokasi, kamera, dan media.
- Daftar Activity.
- Activity launcher.
- FileProvider untuk akses file foto laporan.
- Konfigurasi theme dan icon aplikasi.

Cara kerjanya:

- Android membaca manifest untuk tahu komponen apa saja yang dimiliki aplikasi.
- `SplashActivity` menjadi entry point karena punya intent-filter MAIN dan LAUNCHER.
- Permission yang tertulis di sini tetap perlu diminta runtime untuk beberapa kasus, misalnya lokasi.

app/src/main/res/values/strings.xml

File ini menyimpan teks aplikasi.

Cara kerjanya:

- Kotlin/XML mengambil teks melalui `R.string.nama_string`.
- Membuat teks lebih mudah dikelola.
- Memudahkan jika nanti aplikasi ingin mendukung banyak bahasa.

app/src/main/res/values/colors.xml

File ini menyimpan warna aplikasi.

Cara kerjanya:

- XML dan Kotlin bisa memakai warna dengan `@color/nama_warna` atau `R.color.nama_warna`.
- Membantu menjaga konsistensi warna UI.

app/src/main/res/values/themes.xml

File ini mendefinisikan theme utama aplikasi.

Cara kerjanya:

- Theme dipasang dari manifest.
- Mengatur tampilan global seperti warna status bar, Material theme, dan style default komponen.

app/src/main/res/values-night/themes.xml

File theme untuk mode malam/dark mode.

Cara kerjanya:

- Android memakai file ini saat device berada di night mode.
- Nilainya bisa berbeda dari `values/themes.xml`.

app/src/main/res/values/styles.xml

File ini berisi style reusable.

Cara kerjanya:

- Style bisa dipakai banyak komponen XML.
- Membantu agar atribut UI tidak diulang-ulang.

app/src/main/res/values/dimens.xml

File ini menyimpan ukuran reusable.

Cara kerjanya:

- XML bisa memakai `@dimen/nama_dimen`.
- Cocok untuk margin, padding, tinggi komponen, dan radius.

app/src/main/res/menu/bottom_nav_menu.xml

File ini mendefinisikan item bottom navigation.

Cara kerjanya:

- `BottomNavigationView` membaca menu ini.

- ID item menu harus sesuai dengan destination di `nav_main.xml` agar `setupNavController()` bisa bekerja.

app/src/main/res/xml/file_paths.xml

File ini dipakai oleh `FileProvider`.

Cara kerjanya:

- Menentukan folder mana yang boleh dibagikan sebagai URI aman.
- Dipakai saat aplikasi mengambil/menyimpan foto laporan gangguan.

app/src/main/res/xml/data_extraction_rules.xml

File aturan backup/data extraction Android.

Cara kerjanya:

- Dipakai sistem Android untuk menentukan data apa yang boleh diekstrak saat backup/migrasi.

app/src/main/res/xml/backup_rules.xml

File aturan backup aplikasi.

Cara kerjanya:

- Mengatur data apa saja yang ikut backup otomatis Android.
-

Navigation XML

app/src/main/res/navigation/nav_auth.xml

File ini mendefinisikan navigasi auth.

Isinya:

- `LoginFragment`.
- `RegisterFragment`.
- Action dari login ke register dan sebaliknya.

Cara kerjanya:

- `AuthActivity` menyediakan `NavHostFragment`.
- Saat user klik link register, `findNavController().navigate(...)` menjalankan action di file ini.

app/src/main/res/navigation/nav_main.xml

File ini mendefinisikan navigasi utama aplikasi.

Isinya:

- Home.
- Riwayat.
- Status gangguan.
- Profil.
- Detail rute.
- Tambah/edit laporan gangguan.

Cara kerjanya:

- `MainActivity` memasang `BottomNavigationView` ke `NavController`.
- Destination dengan ID yang cocok dengan menu bottom navigation bisa dibuka otomatis.
- Fragment detail bisa dibuka lewat action dari fragment lain.

Activity

`app/src/main/java/com/example/naikapa/presentation/splash/SplashActivity.kt`

Activity pertama yang dibuka saat aplikasi dijalankan.

Cara kerjanya:

1. Inflate `ActivitySplashBinding`.
2. Membuat `SessionManager`.
3. Menunggu sekitar 1,5 detik.
4. Mengecek apakah user sudah login.
5. Jika sudah login, buka `MainActivity`.
6. Jika belum login, buka `AuthActivity`.
7. Menutup splash dengan `finish()`.

File ini penting untuk memahami entry flow aplikasi.

`app/src/main/java/com/example/naikapa/presentation/auth/AuthActivity.kt`

Activity container untuk login dan register.

Cara kerjanya:

- Menampilkan layout auth yang berisi `NavHostFragment`.
- `NavHostFragment` memakai `nav_auth.xml`.
- Activity ini tidak berisi banyak logic bisnis, hanya menjadi wadah fragment auth.

app/src/main/java/com/example/naikapa/MainActivity.kt

Activity utama setelah user login.

Cara kerjanya:

1. Mengaktifkan edge-to-edge display.
2. Inflate `ActivityMainBinding`.
3. Mengambil `NavHostFragment`.
4. Mengambil `NavController`.
5. Menghubungkan `bottomNavigation` dengan `NavController`.
6. Mengatur padding bottom navigation agar tidak tertutup navigation bar.

File ini penting karena menjadi container halaman utama aplikasi.

Fragment Auth

app/src/main/java/com/example/naikapa/presentation/auth/LoginFragment.kt

Fragment untuk login user.

Cara kerjanya:

1. Inflate `FragmentLoginBinding`.
2. Membuat `SessionManager`.
3. Membuat `NaikApaDatabaseHelper`.
4. Membuat `UserDao`.
5. Saat tombol login diklik, validasi email dan password.
6. Memanggil `userDao.login(email, password)`.
7. Jika berhasil, simpan session.
8. Buka `MainActivity`.
9. Tutup `AuthActivity`.

Bagian yang perlu diperhatikan:

- Validasi input terjadi di `Fragment`.
- Pengecekan akun terjadi di `UserDao`.
- Status login disimpan di `SessionManager`.

app/src/main/java/com/example/naikapa/presentation/auth/RegisterFragment.kt

Fragment untuk membuat akun baru.

Cara kerjanya:

1. User mengisi nama, email, password, dan data kendaraan.
2. Fragment melakukan validasi input.
3. Data dikirim ke UserDao.
4. Jika email belum terdaftar, user baru disimpan ke database.
5. User diarahkan ke login atau langsung masuk sesuai implementasi.

File ini penting untuk memahami proses insert user ke SQLite.

Fragment Utama

app/src/main/java/com/example/naikapa/presentation/home/HomeFragment.kt

Ini adalah file paling besar dan paling penting di sisi UI.

Tanggung jawabnya:

- Menampilkan peta.
- Mengatur input asal dan tujuan.
- Meminta permission lokasi.
- Mengambil lokasi GPS.
- Mencari lokasi dari TomTom dan GTFS lokal.
- Mengatur pilihan moda.
- Mengatur pilihan prioritas.
- Memanggil recommendation engine.
- Menampilkan hasil rekomendasi.
- Menggambar marker dan polyline di peta.
- Menyimpan riwayat rute.
- Membuka detail rute.

Cara kerjanya secara besar:

1. `onCreate()` mengatur konfigurasi osmdroid.
2. `onViewCreated()` menginisialisasi session, database, DAO, repository, recommendation engine, adapter, peta, chip, search, dan tombol.

3. User memilih asal dan tujuan.
4. User memilih moda dan prioritas.
5. User menekan tombol cari rute.
6. Fragment mengirim input ke RecommendationEngine.
7. Hasil rekomendasi ditampilkan di RecyclerView.
8. Jika item diklik, data rute disimpan di RouteDetailSharedState, lalu pindah ke RouteDetailFragment.

Catatan belajar:

- Jangan baca file ini dari atas sampai bawah sekaligus.
- Pecah menjadi bagian: setup, search, location, route action, map drawing, history, lifecycle.

app/src/main/java/com/example/naikapa/presentation/ route_detail/RouteDetailFragment.kt

Fragment untuk menampilkan detail rute yang dipilih.

Cara kerjanya:

1. Mengambil rute dari RouteDetailSharedState.
2. Menampilkan ringkasan skor, durasi, biaya, dan warning.
3. Menampilkan langkah perjalanan dalam timeline.
4. Menampilkan peta/detail tambahan jika tersedia.
5. Menyediakan aksi seperti simpan perjalanan jika diimplementasikan di layar ini.

File ini penting untuk memahami bagaimana hasil rekomendasi dibaca setelah user klik item di Home.

app/src/main/java/com/example/naikapa/presentation/history/ RiwayatFragment.kt

Fragment untuk menampilkan riwayat dan perjalanan tersimpan.

Cara kerjanya:

1. Membuka database.
2. Mengambil data dari HistoryDao dan SavedTripDao.
3. Menampilkan data memakai adapter RecyclerView.
4. Saat user memilih riwayat, membuat HistoryReplayRequest.
5. Mengarahkan user kembali ke Home agar pencarian bisa diulang.

app/src/main/java/com/example/naikapa/presentation/report/ StatusGangguanFragment.kt

Fragment untuk daftar laporan gangguan.

Cara kerjanya:

1. Mengambil laporan gangguan dari `DisruptionReportDao`.
2. Menampilkan laporan aktif/expired.
3. Menyediakan aksi tambah, edit, atau hapus laporan.
4. Membuka `AddEditDisruptionReportFragment` jika user ingin menambah/mengubah laporan.

`app/src/main/java/com/example/naikapa/presentation/report/AddEditDisruptionReportFragment.kt`

Fragment untuk membuat atau mengedit laporan gangguan.

Cara kerjanya:

1. User memilih kategori gangguan.
2. User mengisi deskripsi.
3. User bisa mengambil foto dari kamera/galeri.
4. Fragment memvalidasi input.
5. Data disimpan lewat `DisruptionReportDao`.
6. Laporan diberi waktu expired sesuai konstanta aplikasi.

`app/src/main/java/com/example/naikapa/presentation/profile/ProfilFragment.kt`

Fragment untuk profil user.

Cara kerjanya:

1. Mengambil user aktif dari `SessionManager`.
2. Mengambil detail user/profil dari database lewat `UserDao`.
3. Menampilkan data profil.
4. Menyimpan perubahan profil jika user mengedit.
5. Mengupdate session jika nama/email berubah.
6. Menangani logout.

Layout XML

`app/src/main/res/layout/activity_splash.xml`

Layout untuk layar splash.

Cara kerjanya:

- Di-inflate oleh `SplashActivity`.

- Biasanya menampilkan logo dan branding aplikasi.

app/src/main/res/layout/activity_auth.xml

Layout container untuk flow auth.

Cara kerjanya:

- Di-inflate oleh AuthActivity.
- Berisi NavHostFragment yang memakai nav_auth.xml.

app/src/main/res/layout/activity_main.xml

Layout container utama aplikasi.

Cara kerjanya:

- Di-inflate oleh MainActivity.
- Berisi NavHostFragment untuk halaman utama.
- Berisi BottomNavigationView.

app/src/main/res/layout/fragment_login.xml

Layout form login.

Cara kerjanya:

- Di-inflate oleh LoginFragment.
- Berisi input email, input password, tombol login, dan link register.
- View diakses dari Kotlin lewat FragmentLoginBinding.

app/src/main/res/layout/fragment_register.xml

Layout form register.

Cara kerjanya:

- Di-inflate oleh RegisterFragment.
- Berisi input data user baru.
- Data dari input dipakai untuk membuat akun lokal.

app/src/main/res/layout/fragment_home.xml

Layout layar utama pencarian rute.

Cara kerjanya:

- Di-inflate oleh HomeFragment.
- Berisi peta, input asal/tujuan, pilihan moda, pilihan prioritas, tombol lokasi, tombol cari rute, dan list rekomendasi.
- Banyak ID di layout ini diakses langsung dari HomeFragment.

app/src/main/res/layout/fragment_route_detail.xml

Layout detail rute.

Cara kerjanya:

- Di-inflate oleh `RouteDetailFragment`.
- Menampilkan detail rute dan timeline langkah perjalanan.

app/src/main/res/layout/fragment_riwayat.xml

Layout riwayat.

Cara kerjanya:

- Di-inflate oleh `RiwayatFragment`.
- Menampilkan riwayat pencarian, riwayat rute, dan/atau perjalanan favorit.

app/src/main/res/layout/fragment_status_gangguan.xml

Layout daftar status gangguan.

Cara kerjanya:

- Di-inflate oleh `StatusGangguanFragment`.
- Menampilkan list laporan gangguan dan tombol tambah laporan.

app/src/main/res/layout/fragment_add_edit_disruption_report.xml

Layout tambah/edit laporan gangguan.

Cara kerjanya:

- Di-inflate oleh `AddEditDisruptionReportFragment`.
- Berisi input kategori, deskripsi, foto, dan tombol simpan.

app/src/main/res/layout/fragment_profil.xml

Layout profil user.

Cara kerjanya:

- Di-inflate oleh `ProfilFragment`.
- Menampilkan dan mengedit data user.

app/src/main/res/layout/item_route_recommendation.xml

Layout satu item rekomendasi rute.

Cara kerjanya:

- Dipakai oleh `RouteResultAdapter`.
- Setiap `ScoredRoute` akan ditampilkan memakai layout ini.

app/src/main/res/layout/item_route_step.xml

Layout satu langkah perjalanan.

Cara kerjanya:

- Dipakai oleh `RouteStepAdapter`.
- Menampilkan instruksi seperti jalan kaki, naik kendaraan, transit, atau turun.

app/src/main/res/layout/item_route_history.xml

Layout satu item riwayat rute.

Cara kerjanya:

- Dipakai oleh `RouteHistoryAdapter`.
- Menampilkan ringkasan asal, tujuan, mode, prioritas, skor, dan waktu.

app/src/main/res/layout/item_search_history.xml

Layout satu item riwayat pencarian lokasi.

Cara kerjanya:

- Dipakai oleh `SearchHistoryAdapter`.
- Menampilkan keyword/lokasi yang pernah dicari.

app/src/main/res/layout/item_saved_trip.xml

Layout satu item perjalanan favorit.

Cara kerjanya:

- Dipakai oleh `SavedTripAdapter`.
- Menampilkan nama perjalanan, asal, tujuan, mode, dan catatan.

app/src/main/res/layout/item_disruption_report.xml

Layout satu item laporan gangguan.

Cara kerjanya:

- Dipakai oleh `DisruptionReportAdapter`.
- Menampilkan kategori, deskripsi, status, waktu, dan foto jika ada.

Common Helper

app/src/main/java/com/example/naikapa/common/AppConstants.kt

File ini menyimpan konstanta aplikasi.

Contoh isi:

- Nama database.

- Kecepatan jalan kaki.
- Radius transfer jalan kaki.
- Konfigurasi TomTom.
- Konfigurasi peta.
- Bobot scoring rekomendasi.
- Tarif transportasi.
- Kategori laporan gangguan.

Cara kerjanya:

- File lain mengambil nilai dari sini supaya angka/string penting tidak tersebar.
- Misalnya `RecommendationScorer` memakai bobot scoring dari file ini.
- `HomeFragment` memakai konfigurasi map dan search dari file ini.

app/src/main/java/com/example/naikapa/common/SessionManager.kt

File ini mengelola session login user.

Cara kerjanya:

- Memakai `SharedPreferences`.
- Menyimpan `isLoggedIn`, `userId`, `userName`, dan `userEmail`.
- Dipakai oleh `SplashActivity` untuk menentukan tujuan setelah splash.
- Dipakai oleh fragment lain untuk tahu user aktif.
- `logout()` membersihkan session.

app/src/main/java/com/example/naikapa/common/ViewExtensions.kt

File ini berisi extension/helper untuk View.

Cara kerjanya:

- Menambahkan fungsi utilitas agar kode UI lebih ringkas.
- Contoh umum: helper `Toast`, padding status bar, margin status bar, atau show/hide view.
- Dipakai oleh fragment/activity agar tidak mengulang kode kecil berkali-kali.

Database Local

app/src/main/java/com/example/naikapa/data/local/NaikApaDbContract.kt

File ini adalah kontrak database.

Isinya:

- Nama tabel.
- Nama kolom.
- SQL CREATE TABLE.
- SQL index.
- SQL drop table.

Cara kerjanya:

- DAO memakai konstanta dari file ini agar nama tabel/kolom konsisten.
- `NaikApaDatabaseHelper` memakai `createTableStatements` saat database dibuat.
- Jika nama kolom berubah, file ini adalah tempat utama yang harus dicek.

***app/src/main/java/com/example/naikapa/data/local/
NaikApaDatabaseHelper.kt***

File ini mengelola lifecycle database SQLite.

Cara kerjanya:

1. Mewarisi `SQLiteOpenHelper`.
2. Memastikan database prebuilt GTFS disalin lewat `PrebuiltDatabaseCopier`.
3. Mengaktifkan foreign key.
4. Membuat tabel dan index di `onCreate()`.
5. Menghapus dan membuat ulang tabel di `onUpgrade()`.

Catatan:

- Project ini memakai SQLite native, bukan Room.
- Helper ini menjadi pintu masuk DAO ke database.

***app/src/main/java/com/example/naikapa/data/local/
PrebuiltDatabaseCopier.kt***

File ini bertugas menyalin database GTFS dari assets ke internal database app.

Cara kerjanya:

1. Cek apakah file database internal sudah ada.
2. Jika belum ada, buka asset `databases/naikapa_gtfs.db`.
3. Salin file tersebut ke lokasi database internal Android.
4. Kembalikan nama database agar dipakai oleh `SQLiteOpenHelper`.

Kenapa diperlukan:

- File di `assets/` bersifat read-only.
- SQLite perlu file database di lokasi internal app agar bisa dibuka normal.

app/src/main/java/com/example/naikapa/data/local/UserDao.kt

DAO untuk tabel user dan profil.

Cara kerjanya:

- Insert user saat register.
- Login dengan email dan password.
- Mengambil user berdasarkan ID.
- Update data user/profil.
- Mengelola data kendaraan user seperti punya motor/mobil.

Dipakai oleh:

- LoginFragment.
- RegisterFragment.
- ProfilFragment.

app/src/main/java/com/example/naikapa/data/local/GtfsDao.kt

DAO untuk data GTFS.

Cara kerjanya:

- Mengambil stop.
- Mencari stop berdasarkan nama.
- Mengambil route.
- Mengambil koneksi stop berurutan.
- Menghitung jumlah data GTFS.
- Mengambil stop untuk pencarian terdekat.

Dipakai oleh:

- GtfsStopSearchRepository.
- NearbyTransitStopRepository.
- TransitGraphRepository.

app/src/main/java/com/example/naikapa/data/local/HistoryDao.kt

DAO untuk riwayat.

Cara kerjanya:

- Menyimpan riwayat pencarian lokasi.
- Menyimpan riwayat rekomendasi rute.
- Mengambil riwayat berdasarkan user.
- Menghapus riwayat jika tersedia.

Dipakai oleh:

- HomeFragment saat menyimpan riwayat.
- RiwayatFragment saat menampilkan riwayat.

app/src/main/java/com/example/naikapa/data/local/SavedTripDao.kt

DAO untuk perjalanan favorit.

Cara kerjanya:

- Menyimpan rute favorit.
- Mengambil daftar favorit user.
- Menghapus favorit.
- Mengubah catatan/nama perjalanan jika tersedia.

Dipakai oleh:

- RiwayatFragment.
- Kemungkinan RouteDetailFragment saat user menyimpan rute.

app/src/main/java/com/example/naikapa/data/local/DisruptionReportDao.kt

DAO untuk laporan gangguan.

Cara kerjanya:

- Insert laporan baru.
- Update laporan.
- Delete laporan.
- Ambil semua laporan.
- Ambil laporan aktif.
- Menentukan laporan expired berdasarkan expired_at.

Dipakai oleh:

- StatusGangguanFragment.
- AddEditDisruptionReportFragment.
- RecommendationEngine untuk memberi penalti skor pada rute terdampak.

app/src/main/java/com/example/naikapa/data/local/RouteCacheDao.kt

DAO untuk cache rute.

Cara kerjanya:

- Menyimpan hasil rute dalam bentuk JSON.
- Mengambil cache berdasarkan asal, tujuan, mode, dan prioritas.
- Mengambil cache langsung berdasarkan `cache_key` lewat `findByKey`.
- Menyimpan cache baru atau mengganti cache lama dengan `insertOrReplace`.
- Menghapus cache lama dengan `clearOlderThan`.
- Menghapus semua cache dengan `clearAll`.
- Membantu menghindari hitung/API ulang jika data masih relevan.

Status integrasi:

- `RouteCacheDao` sudah dipakai oleh `RouteCacheRepository`.
- `RouteCacheRepository` sudah dipakai oleh `HomeFragment` pada flow rekomendasi utama.
- DAO tetap sederhana: hanya tahu operasi SQLite, sedangkan aturan TTL, cache key, dan serialize/deserialize ada di repository.

Detail yang perlu dipahami:

- `cache_key` bersifat unik, sehingga satu kombinasi input pencarian hanya punya satu row aktif terbaru.
- `insertOrReplace` memakai conflict replace, jadi cache lama untuk key yang sama diganti hasil baru.
- `created_at` dipakai untuk mengecek apakah cache masih valid dan untuk membersihkan row lama.
- Method `find(...)` berbasis kolom koordinat/mode/priority masih ada, tetapi flow baru lebih stabil memakai `findByKey(...)`.

Model Data

app/src/main/java/com/example/naikapa/data/model/User.kt

Model data user.

Cara kerjanya:

- Mewakili satu baris data dari tabel `users`.
- Dipakai setelah login/register atau saat membaca profil user.

app/src/main/java/com/example/naikapa/data/model/ UserProfile.kt

Model data profil user.

Cara kerjanya:

- Mewakili data tambahan user dari tabel `user_profiles`.
- Berisi preferensi seperti mode default, prioritas default, atau lokasi rumah jika tersedia.

app/src/main/java/com/example/naikapa/data/model/ GtfsModels.kt

Model untuk data GTFS.

Biasanya mencakup:

- Stop.
- Route.
- Trip.
- Stop time.
- Statistik jumlah tabel/agency.
- Koneksi antar stop.

Cara kerjanya:

- `GtfsDao` mengubah `Cursor SQLite` menjadi model-model ini.
- `Repository routing` memakai model ini untuk membangun graph.

app/src/main/java/com/example/naikapa/data/model/ TransitGraphModels.kt

Model untuk graph transit.

Biasanya mencakup:

- Node.
- Edge.
- Graph.
- Metadata route/stop.

Cara kerjanya:

- `TransitGraphBuilder` membuat graph dari data GTFS.
- `DijkstraAlgorithm` membaca graph ini untuk mencari path.

app/src/main/java/com/example/naikapa/data/model/ TransitRouteModels.kt

Model hasil routing transit.

Biasanya mencakup:

- `TransitRouteResult`.
- `RouteStep`.
- Metrik rute.
- Mode transit.
- Sort preference.

Cara kerjanya:

- `TransitRoutingRepository` menghasilkan model ini.
- `RecommendationEngine` mengubahnya menjadi kandidat rekomendasi.
- UI menampilkan langkahnya di detail rute.

***app/src/main/java/com/example/naikapa/data/model/
CombinedRouteModels.kt***

Model untuk rute gabungan.

Cara kerjanya:

- Mewakili perjalanan kendaraan pribadi + transit.
- Berisi segmen kendaraan pribadi, segmen transit, dan metrik gabungan.
- Dihasilkan oleh `CombinedRouteRepository`.

***app/src/main/java/com/example/naikapa/data/model/
RecommendationModels.kt***

Model untuk sistem rekomendasi.

Biasanya mencakup:

- Kandidat rute.
- Rute yang sudah diberi skor.
- Hasil rekomendasi utama dan alternatif.
- Filter kendaraan.

Cara kerjanya:

- `RecommendationEngine` membuat `RecommendationResult`.
- `RouteResultAdapter` menampilkan `ScoredRoute`.

***app/src/main/java/com/example/naikapa/data/model/
TomTomSearchModels.kt***

Model response dari TomTom Search API.

Cara kerjanya:

- Retrofit + Gson mengubah JSON response menjadi data class di file ini.
- Repository kemudian mengubahnya menjadi model internal SearchLocation.

app/src/main/java/com/example/naikapa/data/model/TomTomRoutingModels.kt

Model response dari TomTom Routing API.

Cara kerjanya:

- Mewakili struktur JSON rute TomTom.
- Dipakai oleh TomTomRoutingRepository untuk mengambil durasi, jarak, biaya estimasi, dan polyline jika tersedia.

app/src/main/java/com/example/naikapa/data/model/SearchLocation.kt

Model lokasi hasil pencarian.

Cara kerjanya:

- Bisa berasal dari TomTom atau GTFS lokal.
- Berisi nama, alamat, latitude, longitude, dan metadata seperti stop ID jika ada.
- Dipakai oleh HomeFragment sebagai tujuan atau asal pilihan user.

app/src/main/java/com/example/naikapa/data/model/LocationPoint.kt

Model titik lokasi asal.

Cara kerjanya:

- Dipakai untuk lokasi GPS atau titik asal manual.
- Berisi label, latitude, longitude, dan penanda apakah berasal dari GPS.

app/src/main/java/com/example/naikapa/data/model/MapModels.kt

Model untuk kebutuhan peta.

Cara kerjanya:

- Menyimpan data marker, tipe marker, dan style map.
- Dipakai oleh HomeFragment saat menggambar marker dan polyline.

app/src/main/java/com/example/naikapa/data/model/SearchHistory.kt

Model riwayat pencarian lokasi.

Cara kerjanya:

- Mewakili satu baris tabel search_history.

- Ditampilkan di layar riwayat atau dipakai untuk replay pencarian.

app/src/main/java/com/example/naikapa/data/model/RouteHistory.kt

Model riwayat rute.

Cara kerjanya:

- Mewakili satu rekomendasi rute yang pernah dihitung.
- Berisi origin, destination, mode, priority, skor, estimasi waktu, biaya, dan metrik lain.

app/src/main/java/com/example/naikapa/data/model/SavedTrip.kt

Model perjalanan favorit.

Cara kerjanya:

- Mewakili data di tabel saved_trips.
- Dipakai untuk menampilkan dan membuka ulang perjalanan favorit.

app/src/main/java/com/example/naikapa/data/model/DisruptionReport.kt

Model laporan gangguan.

Cara kerjanya:

- Mewakili satu laporan dari tabel disruption_reports.
- Dipakai oleh UI laporan dan oleh recommendation engine untuk menentukan penalti.

app/src/main/java/com/example/naikapa/data/model/RouteCache.kt

Model cache rute.

Cara kerjanya:

- Mewakili hasil rute yang disimpan sementara dalam database.
- Berisi input pencarian yang sudah dinormalisasi dan JSON hasil rekomendasi.
- Field identitas cache: cacheKey.
- Field input cache: origin latitude/longitude, destination latitude/longitude, mode, dan priority.
- Field output cache: resultJson, hasil serialize dari RecommendationResult.
- Field waktu: createdAt, dipakai untuk menentukan umur cache.
- Model ini aktif dipakai oleh RouteCacheDao dan RouteCacheRepository.

Kenapa hasil disimpan sebagai JSON:

- Hasil rekomendasi berisi kandidat transit, kendaraan pribadi, dan gabungan yang bentuknya

berbeda.

- Menyimpan JSON membuat cache cukup fleksibel tanpa harus membuat banyak tabel detail untuk setiap tipe rute.
 - Trade-off-nya: isi cache tidak cocok untuk query analitik detail, karena JSON dibaca ulang sebagai satu objek rekomendasi.
-

Remote API

app/src/main/java/com/example/naikapa/data/remote/RemoteClient.kt

File ini membuat client Retrofit.

Cara kerjanya:

1. Membuat `HttpLoggingInterceptor` dengan level BASIC.
2. Membuat `OkHttpClient` dan memasang interceptor tersebut.
3. Membuat `Retrofit` dengan `AppConstants.TOMTOM_BASE_URL`.
4. Menambahkan `GsonConverterFactory` supaya JSON TomTom bisa dipetakan ke data class.
5. Membuat instance `TomTomSearchApi` dan `TomTomRoutingApi`.

File ini adalah titik pusat konfigurasi HTTP client.

Catatan:

- File ini tidak menyimpan API key.
- API key dikirim dari repository saat request dipanggil.
- Jika nanti ada timeout, header global, atau interceptor tambahan, tempat utamanya ada di file ini.

app/src/main/java/com/example/naikapa/data/remote/TomTomSearchApi.kt

Interface Retrofit untuk TomTom Search API.

Cara kerjanya:

- Mendefinisikan endpoint `GET search/2/search/{query}.json`.
- query dikirim sebagai path.
- key, countrySet, typeahead, limit, lat, dan lon dikirim sebagai query parameter.
- Return type memakai `Response<TomTomSearchResponse>` agar repository bisa mengecek HTTP status.

- Retrofit membuat implementasinya otomatis saat runtime.

Fungsi file ini hanya kontrak API. Validasi query, validasi API key, dan mapping ke `SearchLocation` dilakukan di `TomTomSearchRepository`.

app/src/main/java/com/example/naikapa/data/remote/ TomTomRoutingApi.kt

Interface Retrofit untuk TomTom Routing API.

Cara kerjanya:

- Mendefinisikan endpoint GET `routing/1/calculateRoute/{from}:{to}/json`.
- `from` dan `to` berisi koordinat `lat`, `lon`.
- `travelMode` menentukan jenis rute: motor, mobil, atau pedestrian.
- `routeType` memakai `fastest`.
- `maxAlternatives` menentukan jumlah alternatif rute.
- `avoid` bersifat opsional, misalnya `tollRoads`.
- Return type memakai `Response<TomTomRoutingResponse>`.

File ini dipakai oleh `TomTomRoutingRepository` untuk rute kendaraan pribadi dan rute jalan kaki TomTom.

Repository

app/src/main/java/com/example/naikapa/data/repository/ TomTomSearchRepository.kt

Repository pencarian lokasi online.

Cara kerjanya:

1. Menerima query dari `HomeFragment`.
2. Trim query dan menolak query yang lebih pendek dari `TOMTOM_MIN_QUERY_LENGTH`.
3. Mengecek API key agar placeholder tidak dikirim ke TomTom.
4. Memanggil `TomTomSearchApi.search`.
5. Mengirim bias koordinat default Jabodetabek atau lokasi user.
6. Mengecek `response.isSuccessful`.
7. Mengubah setiap `TomTomSearchResult` menjadi `SearchLocation`.
8. Mengembalikan hasil dalam bentuk `Result<List<SearchLocation>>`.

Alasan mapping dilakukan di repository:

- Model response TomTom tidak langsung dipakai UI.
- UI hanya butuh `SearchLocation` yang lebih sederhana.
- Repository bisa membuang hasil yang tidak punya posisi/nama valid.

app/src/main/java/com/example/naikapa/data/repository/ TomTomRoutingRepository.kt

Repository routing kendaraan pribadi.

Cara kerjanya:

1. Menerima koordinat asal dan tujuan.
2. Menerima mode kendaraan, misalnya motor atau mobil.
3. Mengecek API key agar request tidak dijalankan dengan placeholder.
4. Mengubah mode internal menjadi travel mode TomTom: `motorcycle` atau `car`.
5. Mengirim `avoid=tollRoads` hanya untuk mobil saat user memilih hindari tol.
6. Memanggil `TomTomRoutingApi.calculateRoute`.
7. Mengambil `lengthInMeters`, `travelTimeInSeconds`, dan `polyline point` dari response.
8. Mengubah response menjadi `PrivateVehicleRouteResult`.
9. Menghitung estimasi BBM lewat `FuelCostCalculator`.

File ini juga punya `calculateWalkingRoute`:

- Memakai `travelMode=pedestrian`.
- Menghasilkan `WalkingRouteResult`.
- Dipakai oleh rute gabungan untuk first-mile/last-mile jalan kaki.
- Jika gagal, caller bisa fallback ke Haversine.

app/src/main/java/com/example/naikapa/data/repository/ GtfsStopSearchRepository.kt

Repository pencarian stop/stasiun lokal.

Cara kerjanya:

1. Menerima keyword, lokasi user opsional, dan filter agency opsional.
2. Menolak keyword yang lebih pendek dari `GTFS_MIN_QUERY_LENGTH`.
3. Memanggil `GtfsDao.searchStops`.
4. Mengambil hasil lebih banyak dari limit akhir, lalu sorting di Kotlin.
5. Mengubah `GtfsStop` menjadi `SearchLocation`.
6. Menghitung jarak dari user jika koordinat user tersedia.

7. Mengurutkan hasil berdasarkan relevansi keyword, jarak, lalu nama.
8. Menambahkan landmark statis seperti UBM Tower dan Alfa Tower.

Repository ini bekerja offline karena semua datanya berasal dari SQLite GTFS lokal.

app/src/main/java/com/example/naikapa/data/repository/ NearbyTransitStopRepository.kt

Repository pencarian stop terdekat.

Cara kerjanya:

1. Mengambil daftar stop dari `GtfsDao.getStopsForNearestSearch`.
2. Menerima koordinat titik acuan, radius maksimum, limit, dan filter agency opsional.
3. Menghitung jarak tiap stop dengan `GeoDistanceCalculator.haversineMeters`.
4. Membuang stop di luar radius.
5. Mengurutkan berdasarkan jarak, lalu nama stop.
6. Mengubah hasil menjadi `CombinedRouteStopCandidate`.
7. Mengembalikan kandidat stop terdekat dalam radius tertentu.

Dipakai oleh:

- `CombinedRouteRepository`.

Alasan repository ini dipisah:

- Pencarian stop terdekat dipakai untuk rute gabungan.
- Logika ranking stop tidak perlu berada di `CombinedRouteRepository`.
- Mudah dites dengan menyuntikkan list stop palsu.

app/src/main/java/com/example/naikapa/data/repository/ TransitGraphRepository.kt

Repository graph transit.

Cara kerjanya:

1. Mengambil koneksi stop berurutan dari `GtfsDao.getAdjacentStopConnections`.
2. Mengambil semua stop dari `GtfsDao.getAllStopsForGraph`.
3. Mengirim data ke `TransitGraphBuilder`.
4. `TransitGraphBuilder` membuat node, edge transit, dan edge walking transfer.
5. Menyimpan graph di memory lewat `cachedGraph`.
6. Menggunakan `synchronized` agar graph tidak dibangun ganda saat dipanggil bersamaan.
7. Menyediakan `clearCache()` jika graph perlu dibangun ulang.

Catatan:

- Cache di sini adalah cache graph di memory, bukan tabel `route_cache`.
- Graph cache mempercepat Dijkstra karena graph GTFS tidak perlu dibuat ulang setiap pencarian.

app/src/main/java/com/example/naikapa/data/repository/TransitRoutingRepository.kt

Repository pencarian rute transit.

Cara kerjanya:

1. Menerima `startStopId`, `endStopId`, `TransitMode`, dan `SortPreference`.
2. Mengambil graph dari `TransitGraphRepository`.
3. Menjalankan `DijkstraAlgorithm.findPath`.
4. Jika path tidak ditemukan, mengembalikan `null`.
5. Mengambil node awal dan akhir dari graph.
6. Mengubah edge hasil path menjadi langkah perjalanan lewat `RouteStepBuilder`.
7. Menghitung metrik lewat `TransitRouteMetricsCalculator`.
8. Menghasilkan `TransitRouteResult`.

Repository ini menjadi batas antara algoritma routing murni dan kebutuhan aplikasi/UI.

app/src/main/java/com/example/naikapa/data/repository/CombinedRouteRepository.kt

Repository rute gabungan.

Cara kerjanya:

1. Mencari stop terdekat dari origin.
2. Mencari stop terdekat dari destination.
3. Membatasi kombinasi stop dengan `COMBINED_ROUTE_MAX_COMBINATIONS`.
4. Jika pakai kendaraan pribadi, hitung rute kendaraan ke stop awal.
5. Jika tidak pakai kendaraan pribadi, hitung first-mile jalan kaki ke stop awal.
6. Hitung rute transit dari stop awal ke stop akhir.
7. Hitung last-mile jalan kaki dari stop akhir ke tujuan.
8. Untuk jalan kaki, coba TomTom pedestrian terlebih dahulu.
9. Jika TomTom pedestrian gagal, fallback ke Haversine dan `WALKING_SECONDS_PER_METER`.
10. Gabungkan segmen private vehicle/walking, transit, dan walking.
11. Gabungkan metrik durasi, biaya, BBM, jalan kaki, dan transit.

12. Urutkan kandidat sesuai `SortPreference`.

13. Mengembalikan kandidat `CombinedRouteResult`.

Hal yang perlu diperhatikan:

- Rute gabungan bisa berarti Motor + Transit, Mobil + Transit, atau Jalan Kaki + Transit.
- `estimatedTotalCost` = `estimatedFare` + `estimatedBbm`.
- Segmen jalan kaki punya biaya 0, tetapi tetap memengaruhi durasi dan skor walking.

app/src/main/java/com/example/naikapa/data/repository/ RouteCacheRepository.kt

Repository untuk cache hasil rekomendasi rute.

Cara kerjanya:

1. Menerima input pencarian yang sama dengan flow rekomendasi: origin, destination, mode transit, prioritas, filter kendaraan, stop ID, dan opsi tol.
2. Membulatkan koordinat origin/destination memakai `ROUTE_CACHE_COORDINATE_PRECISION`.
3. Membentuk `cacheKey` dari semua input yang memengaruhi hasil rute.
4. Mengecek `RouteCacheDao.findByKey(cacheKey)`.
5. Jika cache tidak ada, return `null`.
6. Jika cache ada tetapi umur lebih dari `ROUTE_CACHE_TTL_MILLIS`, return `null`.
7. Jika cache masih valid, deserialize `resultJson` menjadi `RecommendationResult`.
8. Saat hasil baru sukses dihitung, serialize `RecommendationResult` dan simpan lewat `insertOrReplace`.
9. Membersihkan row lama lewat `clearExpired`, dengan batas fisik `ROUTE_CACHE_CLEANUP_AGE_MILLIS`.

Konstanta penting:

- `ROUTE_CACHE_TTL_MILLIS` = `10 * 60 * 1000L`: cache hanya dipakai selama 10 menit.
- `ROUTE_CACHE_CLEANUP_AGE_MILLIS` = `24 * 60 * 60 * 1000L`: row lama dihapus setelah 24 jam.
- `ROUTE_CACHE_COORDINATE_PRECISION` = `5`: koordinat dibulatkan 5 angka desimal.

Perbedaan TTL dan cleanup:

```
0 - 10 menit      -> cache boleh dipakai
> 10 menit        -> cache dianggap stale, rute dihitung ulang
key yang sama     -> row lama diganti hasil baru
```


> 24 jam -> row lama dibersihkan dari SQLite

Jadi cache tidak dipakai terus menerus. Cache bisa diganti ketika pencarian dengan key yang sama berhasil dihitung ulang, dan cache lama dibersihkan setelah melewati umur cleanup.

app/src/main/java/com/example/naikapa/data/repository/RouteCacheSerializer.kt

Serializer untuk menyimpan dan membaca hasil rekomendasi dari cache.

Cara kerjanya:

- Mengubah `RecommendationResult` menjadi JSON memakai Gson.
- Menyimpan tipe kandidat rute agar saat dibaca ulang bisa dikembalikan ke model yang benar.
- Mendukung kandidat TRANSIT, PRIVATE, dan COMBINED.
- Saat JSON rusak atau model tidak cocok, repository menganggap cache tidak valid dan menghitung rute baru.

Kenapa serializer dipisah dari DAO:

- DAO hanya bertanggung jawab ke SQLite.
- Serializer bertanggung jawab ke format JSON.
- Repository menjadi penghubung yang mengatur aturan bisnis cache: key, TTL, cleanup, dan fallback.

Domain Routing

app/src/main/java/com/example/naikapa/domain/routing/TransitGraphBuilder.kt

File ini membangun graph transit dari data GTFS.

Cara kerjanya:

- Node dibuat dari stop/stasiun.
- Edge dibuat dari koneksi antar stop berurutan dalam trip.
- Edge menyimpan informasi route, agency, waktu, jarak, dan biaya dasar.

Graph ini menjadi input untuk Dijkstra.

app/src/main/java/com/example/naikapa/domain/routing/WalkingTransferBuilder.kt

File ini membuat koneksi jalan kaki antar stop yang berdekatan.

Cara kerjanya:

1. Membandingkan jarak antar stop.

2. Jika jarak di bawah radius tertentu, buat edge walking transfer.
3. Durasi jalan kaki dihitung dari jarak dan kecepatan jalan kaki.

Fungsinya:

- Memungkinkan rute pindah antar halte/stasiun berbeda yang dekat secara geografis.

app/src/main/java/com/example/naikapa/domain/routing/DijkstraAlgorithm.kt

File ini menjalankan algoritma Dijkstra.

Cara kerjanya:

1. Mulai dari stop asal.
2. Hitung cost terkecil ke node tetangga.
3. Pilih node dengan cost sementara paling kecil.
4. Ulangi sampai stop tujuan ditemukan.
5. Rekonstruksi path dari asal ke tujuan.

Cost bisa dipengaruhi oleh:

- Durasi.
- Biaya.
- Jalan kaki.
- Transit.
- Penalti transfer.

app/src/main/java/com/example/naikapa/domain/routing/TransitEdgeCostCalculator.kt

File ini menghitung cost edge untuk routing.

Cara kerjanya:

- Menerima edge dan preferensi sorting.
- Mengubah atribut edge menjadi angka cost.
- Cost ini dipakai oleh Dijkstra untuk menentukan jalur terbaik.

app/src/main/java/com/example/naikapa/domain/routing/RouteStepBuilder.kt

File ini mengubah path edge menjadi langkah perjalanan yang mudah dibaca user.

Cara kerjanya:

- Mengelompokkan edge yang berada pada route yang sama.
- Membuat instruksi seperti naik, turun, transit, atau jalan kaki.

- Menghasilkan list RouteStep.

app/src/main/java/com/example/naikapa/domain/routing/TransitRouteMetricsCalculator.kt

File ini menghitung metrik total rute transit.

Cara kerjanya:

- Menjumlahkan durasi.
- Menjumlahkan biaya.
- Menjumlahkan jarak jalan kaki.
- Menghitung jumlah transit.
- Menghasilkan metrik yang dipakai UI dan scoring.

app/src/main/java/com/example/naikapa/domain/routing/FareCalculator.kt

File ini menghitung tarif transportasi.

Cara kerjanya:

- Membedakan tarif berdasarkan agency/operator.
- TransJakarta bisa flat.
- KRL bisa progresif berdasarkan jarak.
- MRT/LRT bisa punya aturan sendiri.
- Nilai tarif mengacu ke konstanta di AppConstants.

app/src/main/java/com/example/naikapa/domain/routing/GeoDistanceCalculator.kt

File ini menghitung jarak antar koordinat.

Cara kerjanya:

- Menerima latitude/longitude dua titik.
- Menghitung jarak geografis, biasanya dengan formula Haversine.
- Dipakai untuk stop terdekat, walking transfer, dan estimasi jarak.

app/src/main/java/com/example/naikapa/domain/routing/GtfsTimeParser.kt

File ini membaca format waktu GTFS.

Cara kerjanya:

- GTFS memakai format jam seperti HH:mm:ss.
- Pada GTFS, jam bisa lebih dari 24 untuk perjalanan lewat tengah malam.

- Parser mengubah waktu menjadi nilai yang bisa dihitung, misalnya detik.

app/src/main/java/com/example/naikapa/domain/routing/ FuelCostCalculator.kt

File ini menghitung estimasi biaya BBM.

Cara kerjanya:

- Menerima jarak kendaraan.
 - Memakai konsumsi BBM motor/mobil dari AppConstants.
 - Menghasilkan estimasi biaya BBM.
-

Domain Recommendation

app/src/main/java/com/example/naikapa/domain/recommendation/ RecommendationEngine.kt

File ini adalah orkestrator utama rekomendasi.

Cara kerjanya:

1. Menerima origin, destination, mode, priority, kepemilikan kendaraan, API key, filter kendaraan, dan stop ID jika ada.
2. Menjalankan beberapa sumber kandidat rute secara paralel:
 - transit,
 - motor,
 - mobil,
 - gabungan motor + transit,
 - gabungan mobil + transit,
 - transit-only dari stop terdekat.
3. Mengumpulkan kandidat yang berhasil.
4. Mengambil laporan gangguan aktif.
5. Memberi skor tiap kandidat dengan RecommendationScorer.
6. Membuat alasan rekomendasi dengan RecommendationReasonBuilder.
7. Mengurutkan berdasarkan skor.
8. Mengembalikan rekomendasi utama dan maksimal dua alternatif.

app/src/main/java/com/example/naikapa/domain/recommendation/ RecommendationScorer.kt

File ini menghitung skor kandidat rute.

Cara kerjanya:

- Menerima RouteCandidate.
- Membaca metrik durasi, biaya, jalan kaki, transit, dan gangguan.
- Menentukan bobot berdasarkan prioritas user.
- Menormalisasi nilai menjadi skor.
- Mengurangi skor jika terkena laporan gangguan aktif.

app/src/main/java/com/example/naikapa/domain/recommendation/RecommendationReasonBuilder.kt

File ini membuat alasan rekomendasi dalam bahasa yang bisa dibaca user.

Cara kerjanya:

- Membaca kandidat rute dan metriknya.
 - Membandingkan dengan kandidat lain.
 - Membuat kalimat seperti kenapa rute ini cocok, murah, cepat, atau minim jalan kaki.
 - Membuat teks warning jika ada gangguan.
-

Adapter dan Shared State

app/src/main/java/com/example/naikapa/presentation/home/RouteResultAdapter.kt

Adapter untuk list rekomendasi di Home.

Cara kerjanya:

- Menerima list ScoredRoute.
- Inflate `item_route_recommendation.xml`.
- Bind data skor, label rank, estimasi waktu, biaya, dan alasan.
- Menjalankan callback saat item diklik.

app/src/main/java/com/example/naikapa/presentation/route_detail/RouteStepAdapter.kt

Adapter untuk timeline langkah rute.

Cara kerjanya:

- Menerima list RouteStep.
- Inflate `item_route_step.xml`.
- Menampilkan instruksi tiap langkah.

app/src/main/java/com/example/naikapa/presentation/ route_detail/RouteDetailSharedState.kt

File ini menyimpan state sementara rute yang dipilih.

Cara kerjanya:

- HomeFragment mengisi selected route.
- RouteDetailFragment membaca selected route.
- Ini dipakai sebagai cara sederhana mengirim data kompleks antar fragment.

Catatan:

- Untuk project produksi, data antar fragment sering dikirim via Safe Args, ViewModel shared, atau repository/cache.

app/src/main/java/com/example/naikapa/presentation/history/ SearchHistoryAdapter.kt

Adapter untuk riwayat pencarian lokasi.

Cara kerjanya:

- Menampilkan list SearchHistory.
- Memberi callback saat item dipilih.

app/src/main/java/com/example/naikapa/presentation/history/ RouteHistoryAdapter.kt

Adapter untuk riwayat rute.

Cara kerjanya:

- Menampilkan list RouteHistory.
- Menampilkan ringkasan perjalanan.
- Bisa memicu replay rute jika item diklik.

app/src/main/java/com/example/naikapa/presentation/history/ SavedTripAdapter.kt

Adapter untuk perjalanan favorit.

Cara kerjanya:

- Menampilkan list SavedTrip.
- Memberi aksi buka ulang atau hapus favorit jika tersedia.

app/src/main/java/com/example/naikapa/presentation/history/ HistoryReplayRequest.kt

File ini menyimpan request sementara untuk membuka ulang rute dari riwayat.

Cara kerjanya:

1. `RiwayatFragment` mengisi `origin/destination/mode/priority` ke object ini.
2. User diarahkan ke `HomeFragment`.
3. `HomeFragment.onResume()` mengecek apakah ada request pending.
4. Jika ada, field pencarian diisi ulang dan pencarian rute bisa dijalankan.
5. Request dibersihkan.

`app/src/main/java/com/example/naikapa/presentation/report/DisruptionReportAdapter.kt`

Adapter untuk daftar laporan gangguan.

Cara kerjanya:

- Menampilkan list `DisruptionReport`.
- Menampilkan kategori, deskripsi, status, dan foto.
- Memberi callback untuk edit/hapus jika tersedia.

`app/src/main/java/com/example/naikapa/presentation/report/ReportPhotoHelper.kt`

Helper untuk foto laporan gangguan.

Cara kerjanya:

- Membuat file foto.
- Membuat URI lewat `FileProvider`.
- Membantu proses ambil foto dari kamera atau galeri.
- Menyimpan path foto agar bisa dimasukkan ke database.

Assets, Data, dan Script

`app/src/main/assets/databases/naikapa_gtfs.db`

Database SQLite prebuilt berisi data GTFS.

Cara kerjanya:

- Disimpan sebagai asset aplikasi.
- Saat app pertama dijalankan, `PrebuiltDatabaseCopier` menyalinnya ke internal database.
- Setelah disalin, `GtfsDao` bisa membaca tabel GTFS dari database tersebut.

`data/gtfs/`

Folder raw GTFS TransJakarta.

Cara kerjanya:

- Berisi file `.txt` seperti `stops.txt`, `routes.txt`, `trips.txt`, dan `stop_times.txt`.
- Diproses oleh script untuk masuk ke SQLite.

data/gtfs-krl/

Folder raw GTFS KRL.

Cara kerjanya:

- Sama seperti GTFS lain, tetapi berisi data Commuter Line.
- Dipakai sebagai input database GTFS gabungan.

data/gtfs-mrt/

Folder raw GTFS MRT Jakarta.

Cara kerjanya:

- Berisi data stop, route, trip, dan jadwal MRT.
- Diproses menjadi tabel GTFS.

data/gtfs-lrt/

Folder raw GTFS LRT.

Cara kerjanya:

- Berisi data LRT yang dipakai aplikasi.
- Diproses ke database prebuilt.

scripts/build_gtfs_db.py

Script untuk membangun database GTFS.

Cara kerjanya:

1. Membaca file raw GTFS dari folder `data/`.
2. Membuat/mengisi tabel GTFS.
3. Menghasilkan database SQLite `naikapa_gtfs.db`.
4. Database hasilnya bisa ditempatkan di `app/src/main/assets/databases/`.

scripts/validate_gtfs_db.py

Script untuk validasi database GTFS.

Cara kerjanya:

- Mengecek apakah tabel penting ada.
- Mengecek jumlah row.
- Mengecek data stop/route/trip/stop time.

- Membantu memastikan database yang dibundel tidak rusak atau kosong.
-

Drawable, Logo, dan Asset Visual

references/logo/logo.png

Logo utama aplikasi untuk dokumentasi/referensi.

Cara kerjanya:

- Tidak otomatis dipakai oleh Android kecuali disalin ke `res/drawable` atau `mipmap`.
- Dipakai README atau referensi desain.

references/logo/logo_textbawah.png

Logo dengan teks di bawah.

Cara kerjanya:

- Dipakai di README dan/atau sebagai referensi splash/branding.

references/logo/logo_textsampling.png

Logo dengan teks di samping.

Cara kerjanya:

- Dipakai sebagai referensi aset brand.

references/ui/splash-screen.png

Screenshot/referensi UI splash screen.

Cara kerjanya:

- Tidak dijalankan aplikasi.
- Dipakai sebagai pembanding desain.

references/ui/login.png

Screenshot/referensi UI login.

Cara kerjanya:

- Dipakai untuk membandingkan layout `fragment_login.xml`.

references/ui/register.png

Screenshot/referensi UI register.

Cara kerjanya:

- Dipakai untuk membandingkan layout `fragment_register.xml`.

references/ui/home.png

Screenshot/referensi UI home.

Cara kerjanya:

- Dipakai untuk membandingkan layout `fragment_home.xml`.

app/src/main/res/drawable/logo.png

Logo yang sudah masuk resource Android.

Cara kerjanya:

- Bisa dipakai di XML dengan `@drawable/logo`.
- Bisa dipakai dari Kotlin dengan `R.drawable.logo`.

app/src/main/res/drawable/logo_textbawah.png

Logo teks bawah yang tersedia untuk UI Android.

Cara kerjanya:

- Biasanya dipakai di splash/login/register jika dibutuhkan.

app/src/main/res/drawable/logo_textsampling.png

Logo teks samping untuk UI Android.

Cara kerjanya:

- Bisa dipakai di header atau halaman yang butuh brand horizontal.

app/src/main/res/drawable/asset1.png

Asset gambar tambahan untuk UI.

Cara kerjanya:

- Dipakai oleh layout jika direferensikan sebagai `@drawable/asset1`.

app/src/main/res/drawable/asset_splashscreen1.png

Asset visual untuk splash screen.

Cara kerjanya:

- Dipakai oleh layout splash jika direferensikan.

app/src/main/res/drawable/asset_splashscreen2.png

Asset visual tambahan untuk splash screen.

Cara kerjanya:

- Dipakai oleh layout splash atau auth sesuai referensi XML.
-

Test

app/src/test/java/com/example/naikapa/DijkstraAlgorithmTest.kt

Unit test untuk algoritma Dijkstra.

Cara kerjanya:

- Membuat graph kecil.
- Menjalankan pencarian path.
- Memastikan path dan cost sesuai ekspektasi.

app/src/test/java/com/example/naikapa/TransitGraphBuilderTest.kt

Unit test untuk pembuatan graph transit.

Cara kerjanya:

- Memberi data koneksi GTFS contoh.
- Memastikan node dan edge terbentuk benar.

app/src/test/java/com/example/naikapa/WalkingTransferBuilderTest.kt

Unit test untuk walking transfer.

Cara kerjanya:

- Membuat beberapa stop dengan jarak tertentu.
- Memastikan edge jalan kaki dibuat hanya jika jarak memenuhi radius.

app/src/test/java/com/example/naikapa/TransitEdgeCostCalculatorTest.kt

Unit test untuk cost edge.

Cara kerjanya:

- Menguji apakah cost berubah sesuai preferensi seperti tercepat, termurah, atau minim jalan kaki.

app/src/test/java/com/example/naikapa/RouteStepBuilderTest.kt

Unit test untuk pembuatan langkah perjalanan.

Cara kerjanya:

- Memberi path edge.
- Memastikan output step mudah dibaca dan sesuai urutan perjalanan.

app/src/test/java/com/example/naikapa/RecommendationScorerTest.kt

Unit test untuk scoring rekomendasi.

Cara kerjanya:

- Membuat kandidat rute contoh.
- Menguji skor berdasarkan prioritas berbeda.
- Menguji penalti gangguan.

app/src/test/java/com/example/naikapa/FareCalculatorTest.kt

Unit test untuk tarif transportasi.

Cara kerjanya:

- Menguji tarif tiap operator.
- Memastikan tarif flat/progresif sesuai aturan.

app/src/test/java/com/example/naikapa/FuelCostCalculatorTest.kt

Unit test untuk biaya BBM.

Cara kerjanya:

- Menguji estimasi biaya motor/mobil berdasarkan jarak.

app/src/test/java/com/example/naikapa/GtfsTimeParserTest.kt

Unit test untuk parser waktu GTFS.

Cara kerjanya:

- Menguji format jam normal.
- Menguji jam GTFS yang bisa lebih dari 24.

app/src/test/java/com/example/naikapa/TomTomRoutingRepositoryTest.kt

Unit test repository routing TomTom.

Cara kerjanya:

- Biasanya memakai mock API.
- Memastikan response API diubah menjadi model internal dengan benar.

app/src/test/java/com/example/naikapa/GtfsStopSearchRepositoryTest.kt

Unit test pencarian stop GTFS.

Cara kerjanya:

- Menguji query pencarian stop.
- Memastikan hasil diubah menjadi `SearchLocation`.

app/src/test/java/com/example/naikapa/ NearbyTransitStopRepositoryTest.kt

Unit test stop terdekat.

Cara kerjanya:

- Menguji perhitungan jarak dan urutan stop terdekat.

app/src/test/java/com/example/naikapa/ TransitRoutingRepositoryTest.kt

Unit test repository routing transit.

Cara kerjanya:

- Menguji integrasi graph, Dijkstra, step builder, dan metrics calculator dalam skala kecil.

app/src/test/java/com/example/naikapa/ CombinedRouteRepositoryTest.kt

Unit test rute gabungan.

Cara kerjanya:

- Menguji gabungan kendaraan pribadi dan transit.
- Memastikan kandidat gabungan punya metrik total yang benar.

app/src/test/java/com/example/naikapa/ NaikApaDatabaseHelperTest.kt

Unit test database helper.

Cara kerjanya:

- Memastikan tabel bisa dibuat.
- Memastikan struktur database sesuai kontrak.

app/src/test/java/com/example/naikapa/ UiResourceContractTest.kt

Unit test kontrak resource UI.

Cara kerjanya:

- Memastikan resource tertentu tersedia.
- Berguna untuk mencegah crash karena ID/string/drawable hilang.

app/src/test/java/com/example/naikapa/ExampleUnitTest.kt

Unit test contoh bawaan/template.

Cara kerjanya:

- Biasanya hanya contoh sederhana.
- Tidak terlalu penting untuk memahami fitur utama.

***app/src/androidTest/java/com/example/naikapa/
NaikApaDatabaseInstrumentedTest.kt***

Instrumented test database di environment Android.

Cara kerjanya:

- Berjalan di emulator/device.
- Menguji database dengan Context Android nyata.

***app/src/androidTest/java/com/example/naikapa/
FavoriteHistoryDaoInstrumentedTest.kt***

Instrumented test untuk favorit dan riwayat.

Cara kerjanya:

- Menguji DAO yang membutuhkan environment Android.
- Memastikan insert/read/delete berjalan di device/emulator.

***app/src/androidTest/java/com/example/naikapa/
ExampleInstrumentedTest.kt***

Instrumented test contoh bawaan/template.

Cara kerjanya:

- Biasanya mengecek package/context aplikasi.
- Tidak terlalu penting untuk fitur utama.

30. Cara Membaca Satu Fitur dari Awal sampai Akhir

Kalau ingin memahami satu fitur, jangan baca per folder. Baca mengikuti alur data.

Contoh fitur: login.

```
fragment_login.xml
|
v
LoginFragment.kt
|
v
UserDao.kt
|
v
NaikApaDbContract.kt
|
v
SessionManager.kt
|
```

↓
MainActivity.kt

Contoh fitur: cari rekomendasi rute.

```
fragment_home.xml
|
v
HomeFragment.kt
|
v
TomTomSearchRepository.kt / GtfsStopSearchRepository.kt
|
v
RecommendationEngine.kt
|
v
TransitRoutingRepository.kt / TomTomRoutingRepository.kt /
CombinedRouteRepository.kt
|
v
DijkstraAlgorithm.kt / RecommendationScorer.kt
|
v
RouteResultAdapter.kt
|
v
RouteDetailFragment.kt
```

Contoh fitur: laporan gangguan.

```
fragment_status_gangguan.xml
|
v
StatusGangguanFragment.kt
|
v
AddEditDisruptionReportFragment.kt
|
v
ReportPhotoHelper.kt
|
v
DisruptionReportDao.kt
|
v
RecommendationEngine.kt
```

Dengan pola ini, kamu akan lebih mudah memahami hubungan antar file daripada membaca semua file secara acak.